

Part II

Sharp SM83 CPU core

Chapter 3

Introduction to the Sharp SM83 CPU core

The CPU core in the Game Boy SoC is a custom Sharp design that hasn't publicly been given a name by either Sharp or Nintendo. However, using old Sharp datasheets and databooks as evidence, the core has been identified to be a Sharp **SM83** CPU core, or at least something that is 100% compatible with it. SM83 is a custom CPU core used in some custom Application Specific Integrated Chips (ASICs) manufactured by Sharp in the 1980s and 1990s.

! Warning

Some sources claim Game Boy uses a “modified” Zilog Z80 or Intel 8080 CPU core. While the SM83 resembles both and has many identical instructions, it can't execute all Z80/8080 programs, and finer details such as timing of instructions often differ.

SM83 is an 8-bit CPU core with a 16-bit address bus. The Instruction Set Architecture (ISA) is based on both Z80 and 8080, and is close enough to Z80 that programmers familiar with Z80 assembly can quickly become productive with SM83 as well. Some Z80 programs may also work directly on SM83, assuming only opcodes supported by both are used and the program is not sensitive to timing differences.

+ Speculation

Sharp most likely designed SM83 to closely resemble Z80, so it would be easy for programmers already familiar with the widely popular Z80 to write programs for it. However, SM83 is not a “modified Z80” because the internal implementation is completely different. At the time Sharp also manufactured real Z80 chips such as LH0080 under a license from Zilog, so they were familiar with Z80 internals but did not directly copy the actual implementation of the CPU core. If you compare photos of a decapped Z80 chip and a GB SoC, you will see two very different-looking CPU cores.

3.1 History

The first known mention of the SM83 CPU core is in Sharp Microcomputers Data Book (1990), where it is listed as the CPU core used in the SM8320 8-bit microcomputer chip, intended for inverter air conditioners [1]. The data book describes some details of the CPU core, such as a high-level overview of the supported instructions, but precise details such as full opcode tables are not included. Another CPU core called SM82 is also mentioned, but based on the details it's clearly a completely different one.

The SM83 CPU core later appeared in Sharp Microcomputer Data Book (1996), where it is listed as the CPU core in the SM8311/SM8313/SM8314/SM8315 8-bit microcomputer chips, meant for home appliances [2]. This data book describes the CPU core in much more detailed manner, and other than some mistakes in the descriptions, the details seem to match what is known about the GB SoC CPU core from other sources.

Chapter 4

Simple CPU core model

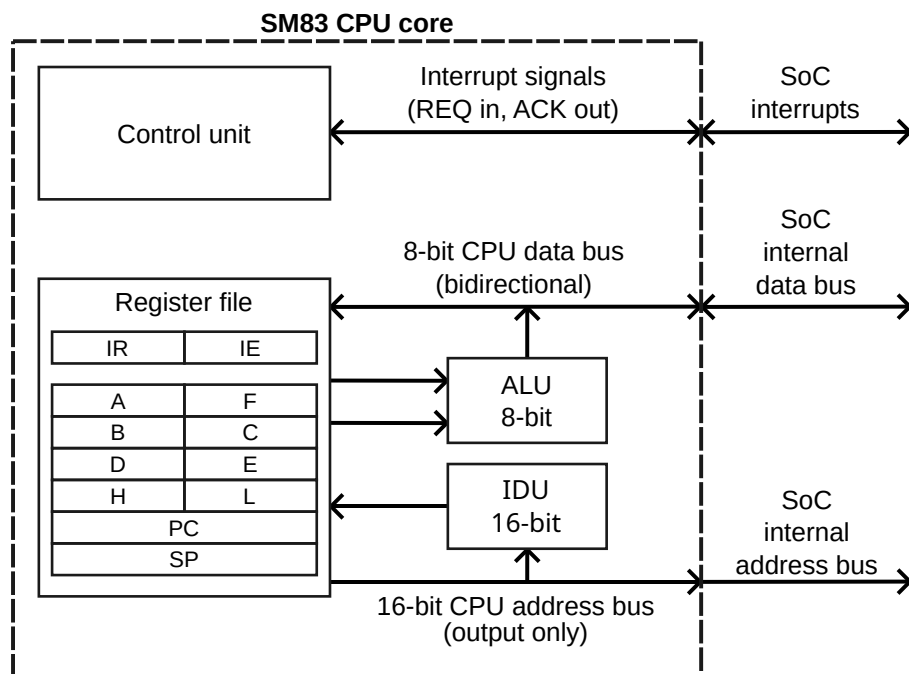


Figure 4.3: Simple model of the SM83 CPU core

Figure 4.3 shows a simplified model of the SM83 CPU core. The core interacts with the rest of the SoC using interrupt signals, an 8-bit bidirectional data bus, and a 16-bit address bus controlled by the CPU core.

The main subsystems of the CPU core are as follows:

Control unit The control unit decodes the executed instructions and generates control signals for the rest of the CPU core. It is also responsible for checking and dispatching interrupts.

Register file The register file holds most of the state of the CPU inside registers. It contains the 16-bit Program Counter (PC), the 16-bit Stack Pointer (SP), the 8-bit Accumulator (A), the Flags register (F), general-purpose register pairs consisting of two 8-bit halves such as BC, DE, HL, and the special-purpose 8-bit registers Instruction Register (IR) and Interrupt Enable (IE).

ALU An 8-bit Arithmetic Logic Unit (ALU) has two 8-bit input ports and is capable of performing various calculations. The ALU outputs its result either to the register file or the CPU data bus.

IDU A dedicated 16-bit Increment/Decrement Unit (IDU) is capable of performing only simple increment/decrement operations on the 16-bit address bus value, but they can be performed independently of the ALU, improving maximum performance of the CPU core. The IDU always outputs its result back to the register file, where it can be written to a register pair or a 16-bit register.

Chapter 5

CPU core timing

5.1 Fetch/execute overlap

Sharp SM83 uses a microprocessor design technique known as *fetch/execute overlap* to improve CPU performance by doing opcode fetches in parallel with instruction execution whenever possible. Since the CPU can only perform one memory access per M-cycle, it is worth it to try to do memory operations as soon as possible. Also, when doing a memory read, the CPU cannot use the data during the same M-cycle so the true minimum effective duration of instructions is 2 machine cycles, not 1 machine cycle.

Every instruction needs one machine cycle for the fetch stage, and at least one machine cycle for the decode/execute stage. However, the fetch stage of an instruction always overlaps with the last machine cycle of the execute stage of the previous instruction. The overlapping execute stage cycle may still do some work (e.g. ALU operation and/or register writeback) but memory access is reserved for the fetch stage of the next instruction.

Since all instructions effectively last one machine cycle longer, fetch/execute overlap is usually ignored in documentation intended for programmers. It is much easier to think of a program as a sequence of non-overlapping instructions and consider only the execute stages when calculating instruction durations. However, when emulating a SM83 CPU core, understanding and emulating the overlap can be useful.

! Warning

Sharp SM831x is a family of single-chip SoCs from Sharp that use the SM83 CPU core, and their datasheet [3] includes a description of fetch/execute overlap. However, the description is not completely correct and can in fact be misleading.

For example, the timing diagram includes an instruction that does not involve opcode fetch at all, and memory operations for two instructions are shown to happen at the same time, which is not possible.

Fetch/execute overlap timing example

Let's assume the CPU is executing a program that starts from the address 0x1000 and contains the following instructions:

0x1000	INC A
0x1001	LDH (n), A
0x1003	RST 0x08
0x0008	NOP

The following timing diagram shows all memory operations done by the CPU, and the fetch and execute stages of each instruction:

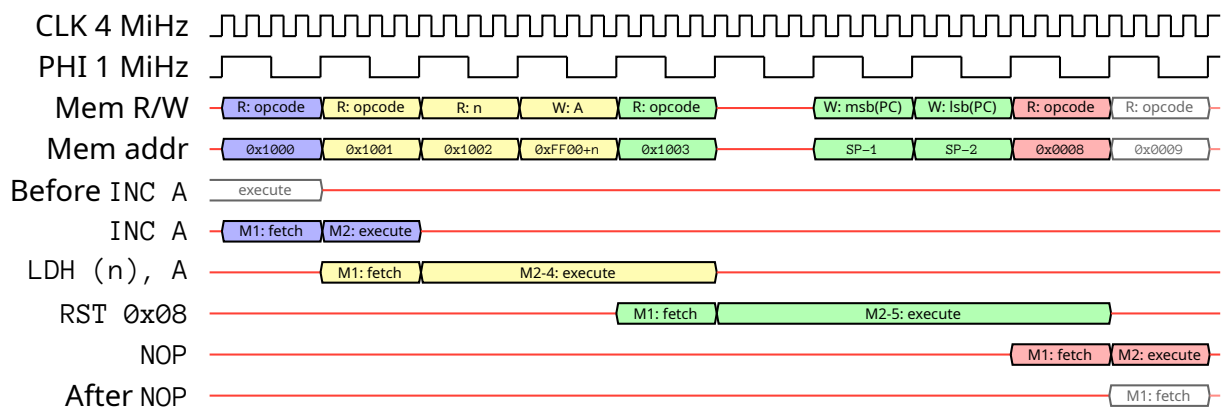


Figure 5.4: Fetch/execute overlap example

Chapter 6

Sharp SM83 instruction set

6.1 Overview

CB opcode prefix

Undefined opcodes

6.2 8-bit load instructions

LD r, r': Load register (register)

Load to the 8-bit register r , data from the 8-bit register r' .

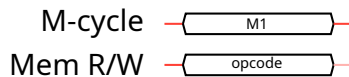
Opcode 0b01xxxxyyy/variou

Duration 1 machine cycle

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

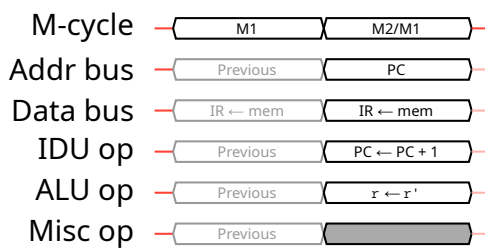


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x41: # example: LD B, C
```

```
    B = C
```

Detailed timing and pseudocode



```
# M2/M1
```

```
if IR == 0x41: # example: LD B, C
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; B = C
```

LD r, n: Load register (immediate)

Load to the 8-bit register r , the immediate data n .

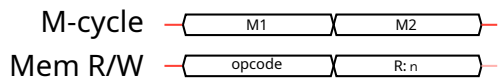
Opcode 0b00xxx110/variou

Duration 2 machine cycles

Length 2 bytes: opcode + n

Flags -

Simple timing and pseudocode

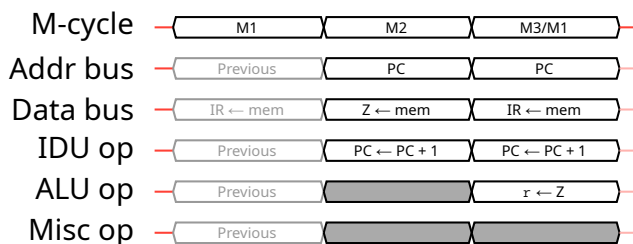


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x06: # example: LD B, n
```

```
    B = read_memory(addr=PC); PC = PC + 1
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x06: # example: LD B, n
```

```
    Z = read_memory(addr=PC); PC = PC + 1
```

```
# M3/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; B = Z
```


LD r, (HL): Load register (indirect HL)

Load to the 8-bit register r , data from the absolute address specified by the 16-bit register HL.

Opcode 0b01xxx110/variou

Duration 2 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

M-cycle 

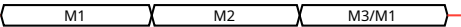
Mem R/W 

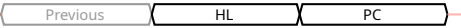
```
opcode = read_memory(addr=PC); PC = PC + 1
```

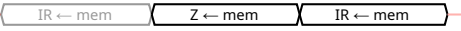
```
if opcode == 0x46: # example: LD B, (HL)
```

```
  B = read_memory(addr=HL)
```

Detailed timing and pseudocode

M-cycle 

Addr bus 

Data bus 

IDU op 

ALU op 

Misc op 

```
# M2
```

```
if IR == 0x46: # example: LD B, (HL)
```

```
  Z = read_memory(addr=HL)
```

```
# M3/M1
```

```
  IR, intr = fetch_cycle(addr=PC); PC = PC + 1; B = Z
```

LD (HL), r: Load from register (indirect HL)

Load to the absolute address specified by the 16-bit register HL, data from the 8-bit register r.

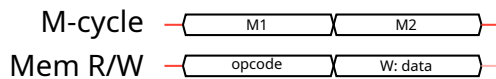
Opcode 0b01110xxx/xxxx/xxxx/xxxx

Duration 2 machine cycles

Length 1 byte: opcode

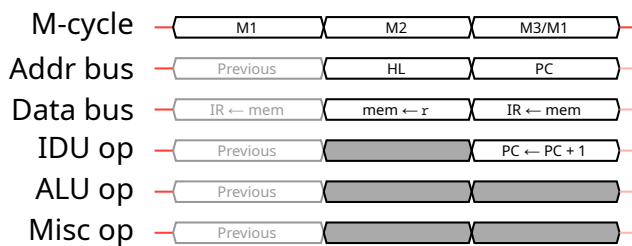
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x70: # example: LD (HL), B
    write_memory(addr=HL, data=B)
```

Detailed timing and pseudocode



```
# M2
if IR == 0x70: # example: LD (HL), B
    write_memory(addr=HL, data=B)
# M3/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LD (HL), n: Load from immediate data (indirect HL)

Load to the absolute address specified by the 16-bit register HL, the immediate data n.

Opcode 0b00110110/0x36

Duration 3 machine cycles

Length 2 bytes: opcode + n

Flags -

Simple timing and pseudocode



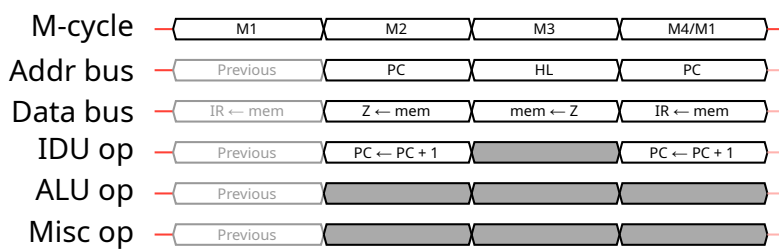
```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x36:
```

```
    n = read_memory(addr=PC); PC = PC + 1
```

```
    write_memory(addr=HL, data=n)
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x36:
```

```
    Z = read_memory(addr=PC); PC = PC + 1
```

```
# M3
```

```
    write_memory(addr=HL, data=Z)
```

```
# M4/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LD A, (BC): Load accumulator (indirect BC)

Load to the 8-bit A register, data from the absolute address specified by the 16-bit register BC.

Opcode 0b00001010/0x0A

Duration 2 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

M-cycle

Mem R/W

```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x0A:
```

```
    A = read_memory(addr=BC)
```

Detailed timing and pseudocode

M-cycle

Addr bus

Data bus

IDU op

ALU op

Misc op

```
# M2
```

```
if IR == 0x0A:
```

```
    Z = read_memory(addr=BC)
```

```
# M3/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; A = Z
```

LD A, (DE): Load accumulator (indirect DE)

Load to the 8-bit A register, data from the absolute address specified by the 16-bit register DE.

Opcode 0b00011010/0x1A

Duration 2 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

M-cycle 

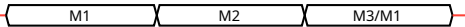
Mem R/W 

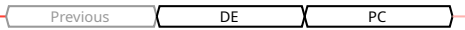
```
opcode = read_memory(addr=PC); PC = PC + 1
```

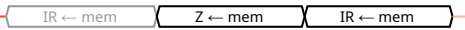
```
if opcode == 0x1A:
```

```
    A = read_memory(addr=DE)
```

Detailed timing and pseudocode

M-cycle 

Addr bus 

Data bus 

IDU op 

ALU op 

Misc op 

```
# M2
```

```
if IR == 0x1A:
```

```
    Z = read_memory(addr=DE)
```

```
# M3/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; A = Z
```

LD (BC), A: Load from accumulator (indirect BC)

Load to the absolute address specified by the 16-bit register BC, data from the 8-bit A register.

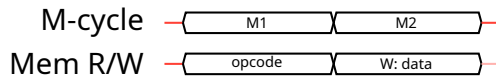
Opcode 0b00000010/0x02

Duration 2 machine cycles

Length 1 byte: opcode

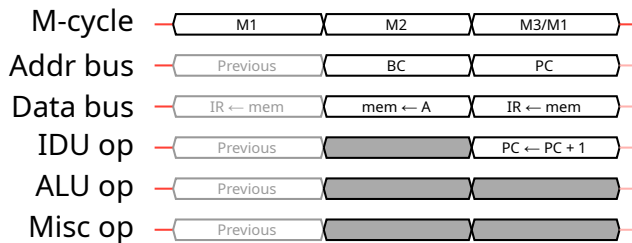
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x02:
    write_memory(addr=BC, data=A)
```

Detailed timing and pseudocode



```
# M2
if IR == 0x02:
    write_memory(addr=BC, data=A)
# M3/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LD (DE), A: Load from accumulator (indirect DE)

Load to the absolute address specified by the 16-bit register DE, data from the 8-bit A register.

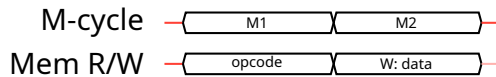
Opcode 0b00010010/0x12

Duration 2 machine cycles

Length 1 byte: opcode

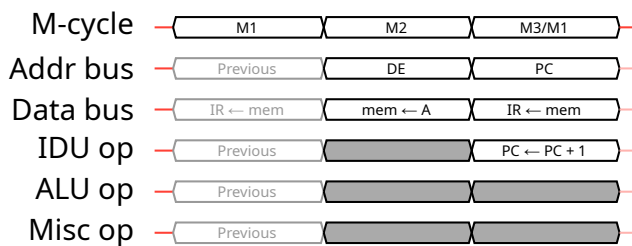
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x12:
    write_memory(addr=DE, data=A)
```

Detailed timing and pseudocode



```
# M2
if IR == 0x12:
    write_memory(addr=DE, data=A)
# M3/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LD A, (nn): Load accumulator (direct)

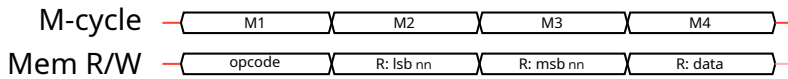
Load to the 8-bit A register, data from the absolute address specified by the 16-bit operand nn.

Opcode 0b11111010/0xFA

Duration 4 machine cycles

Length 3 bytes: opcode + LSB(nn) + MSB(nn) **Flags** -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0xFA:
```

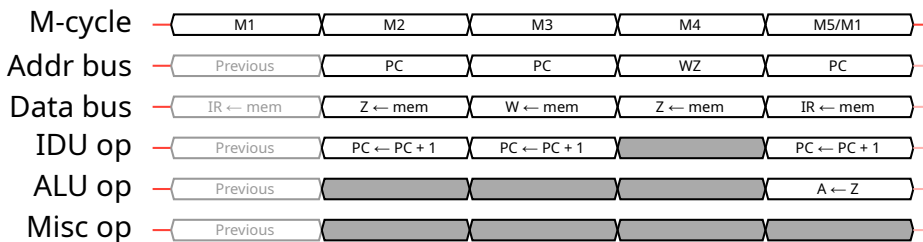
```
    nn_lsb = read_memory(addr=PC); PC = PC + 1
```

```
    nn_msb = read_memory(addr=PC); PC = PC + 1
```

```
    nn = unsigned_16(lsb=nn_lsb, msb=nn_msb)
```

```
    A = read_memory(addr=nn)
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0xFA:
```

```
    Z = read_memory(addr=PC); PC = PC + 1
```

```
# M3
```

```
    W = read_memory(addr=PC); PC = PC + 1
```

```
# M4
```

```
    Z = read_memory(addr=WZ)
```

```
# M5/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; A = Z
```


LD (nn), A: Load from accumulator (direct)

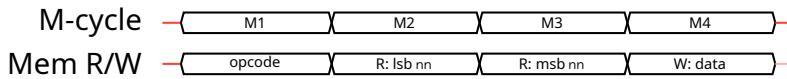
Load to the absolute address specified by the 16-bit operand nn, data from the 8-bit A register.

Opcode 0b11101010/0xEA

Duration 4 machine cycles

Length 3 bytes: opcode + LSB(nn) + MSB(nn) **Flags** -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0xEA:
```

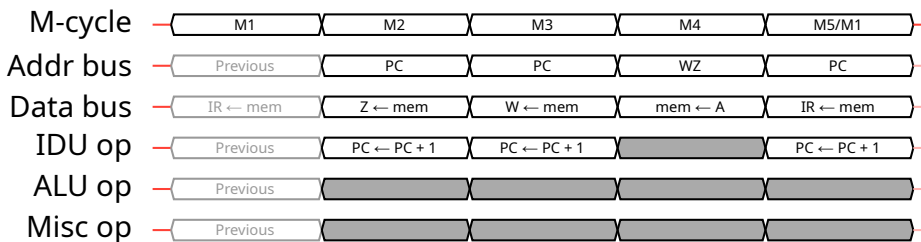
```
    nn_lsb = read_memory(addr=PC); PC = PC + 1
```

```
    nn_msb = read_memory(addr=PC); PC = PC + 1
```

```
    nn = unsigned_16(lsb=nn_lsb, msb=nn_msb)
```

```
    write_memory(addr=nn, data=A)
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0xEA:
```

```
    Z = read_memory(addr=PC); PC = PC + 1
```

```
# M3
```

```
    W = read_memory(addr=PC); PC = PC + 1
```

```
# M4
```

```
    write_memory(addr=WZ, data=A)
```

```
# M5/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LDH A, (C): Load accumulator (indirect 0xFF00+C)

Load to the 8-bit A register, data from the address specified by the 8-bit C register. The full 16-bit absolute address is obtained by setting the most significant byte to 0xFF and the least significant byte to the value of C, so the possible range is 0xFF00–0xFFFF.

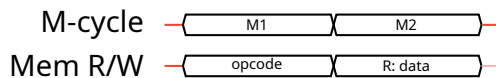
Opcode 0b11110010/0xF2

Duration 2 machine cycles

Length 1 byte: opcode

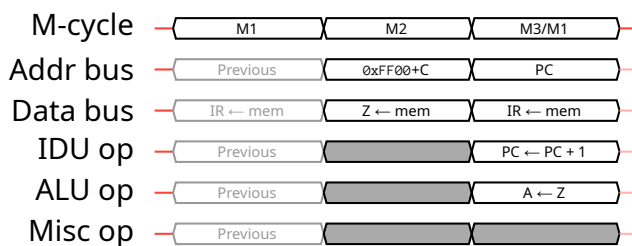
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xF2:
    A = read_memory(addr=unsigned_16(lsb=C, msb=0xFF))
```

Detailed timing and pseudocode



```
# M2
if IR == 0xF2:
    Z = read_memory(addr=unsigned_16(lsb=C, msb=0xFF))
# M3/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1; A = Z
```

LDH (C), A: Load from accumulator (indirect 0xFF00+C)

Load to the address specified by the 8-bit C register, data from the 8-bit A register. The full 16-bit absolute address is obtained by setting the most significant byte to 0xFF and the least significant byte to the value of C, so the possible range is 0xFF00–0xFFFF.

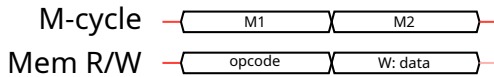
Opcode 0b11100010/0xE2

Duration 2 machine cycles

Length 1 byte: opcode

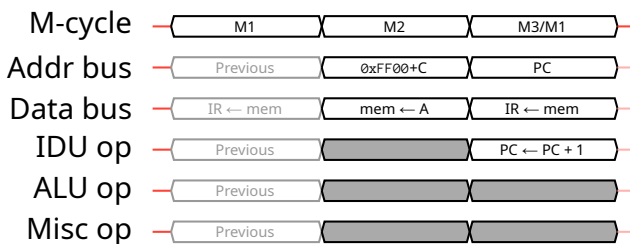
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xE2:
    write_memory(addr=unsigned_16(lsb=C, msb=0xFF), data=A)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xE2:
    write_memory(addr=unsigned_16(lsb=C, msb=0xFF), data=A)
# M3/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LDH A, (n): Load accumulator (direct 0xFF00+n)

Load to the 8-bit A register, data from the address specified by the 8-bit immediate data n. The full 16-bit absolute address is obtained by setting the most significant byte to 0xFF and the least significant byte to the value of n, so the possible range is 0xFF00–0xFFFF.

Opcode 0b11110000/0xF0

Duration 3 machine cycles

Length 2 bytes: opcode + n

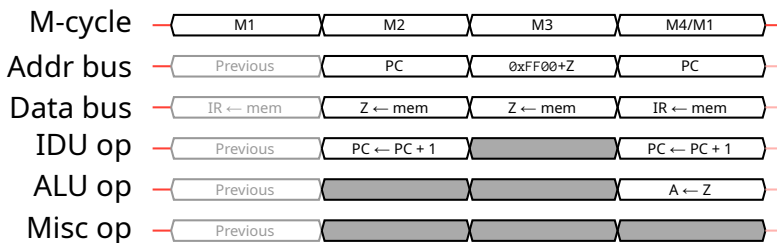
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xF0:
    n = read_memory(addr=PC); PC = PC + 1
    A = read_memory(addr=unsigned_16(lsb=n, msb=0xFF))
```

Detailed timing and pseudocode



```
# M2
if IR == 0xF0:
    Z = read_memory(addr=PC); PC = PC + 1
# M3
Z = read_memory(addr=unsigned_16(lsb=Z, msb=0xFF))
# M4/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1; A = Z
```

LDH (n), A: Load from accumulator (direct 0xFF00+n)

Load to the address specified by the 8-bit immediate data n , data from the 8-bit A register. The full 16-bit absolute address is obtained by setting the most significant byte to 0xFF and the least significant byte to the value of n , so the possible range is 0xFF00–0xFFFF.

Opcode 0b11100000/0xE0

Duration 3 machine cycles

Length 2 bytes: opcode + n

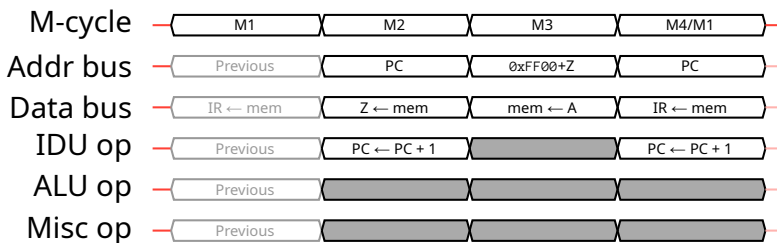
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xE0:
    n = read_memory(addr=PC); PC = PC + 1
    write_memory(addr=unsigned_16(lsb=n, msb=0xFF), data=A)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xE0:
    Z = read_memory(addr=PC); PC = PC + 1
# M3
write_memory(addr=unsigned_16(lsb=Z, msb=0xFF), data=A)
# M4/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LD A, (HL-): Load accumulator (indirect HL, decrement)

Load to the 8-bit A register, data from the absolute address specified by the 16-bit register HL. The value of HL is decremented after the memory read.

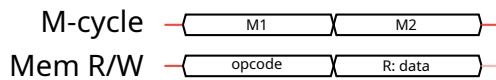
Opcode 0b00111010/0x3A

Duration 2 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

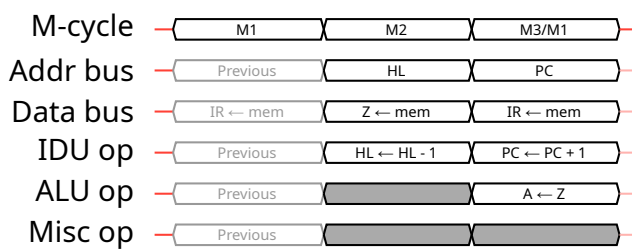


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x3A:
```

```
    A = read_memory(addr=HL); HL = HL - 1
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x3A:
```

```
    Z = read_memory(addr=HL); HL = HL - 1
```

```
# M3/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; A = Z
```

LD (HL-), A: Load from accumulator (indirect HL, decrement)

Load to the absolute address specified by the 16-bit register HL, data from the 8-bit A register. The value of HL is decremented after the memory write.

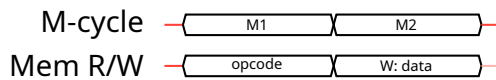
Opcode 0b00110010/0x32

Duration 2 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

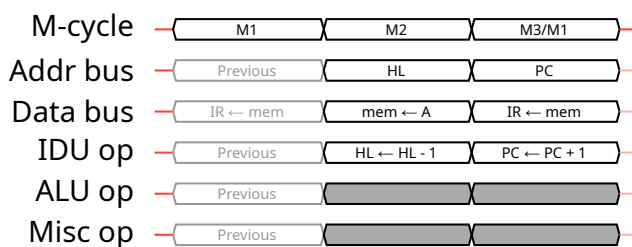


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x32:
```

```
    write_memory(addr=HL, data=A); HL = HL - 1
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x32:
```

```
    write_memory(addr=HL, data=A); HL = HL - 1
```

```
# M3/M1
```

```
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LD A, (HL+): Load accumulator (indirect HL, increment)

Load to the 8-bit A register, data from the absolute address specified by the 16-bit register HL. The value of HL is incremented after the memory read.

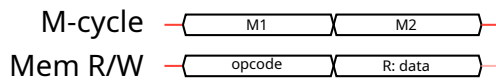
Opcode 0b00101010/0x2A

Duration 2 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

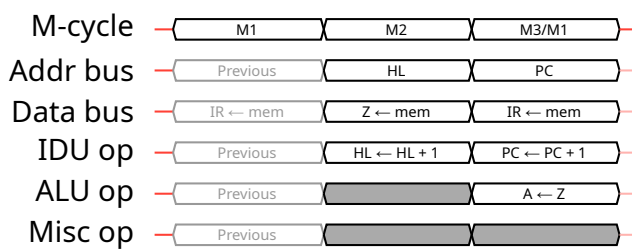


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x2A:
```

```
    A = read_memory(addr=HL); HL = HL + 1
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x2A:
```

```
    Z = read_memory(addr=HL); HL = HL + 1
```

```
# M3/M1
```

```
IR, intr = fetch_cycle(addr=PC); PC = PC + 1; A = Z
```


LD (HL+), A: Load from accumulator (indirect HL, increment)

Load to the absolute address specified by the 16-bit register HL, data from the 8-bit A register. The value of HL is incremented after the memory write.

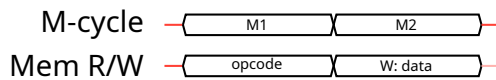
Opcode 0b00100010/0x22

Duration 2 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

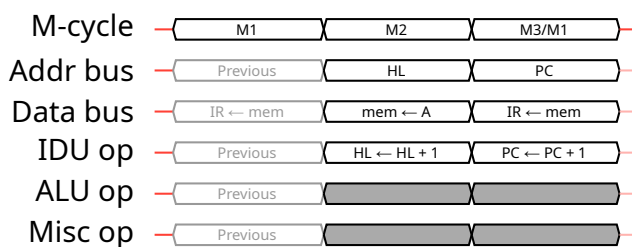


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x22:
```

```
    write_memory(addr=HL, data=A); HL = HL + 1
```

Detailed timing and pseudocode



M2

```
if IR == 0x22:
```

```
    write_memory(addr=HL, data=A); HL = HL + 1
```

M3/M1

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

6.3 16-bit load instructions

LD rr, nn: Load 16-bit register / register pair

Load to the 16-bit register *rr*, the immediate 16-bit data *nn*.

Opcode 0b00xx0001/ various

Duration 3 machine cycles

Length 3 bytes: opcode + LSB(*nn*) + MSB(*nn*)

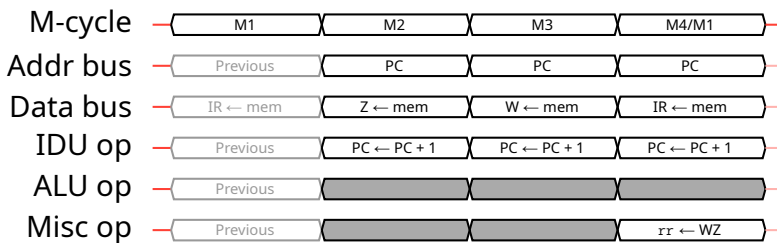
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x01: # example: LD BC, nn
    nn_lsb = read_memory(addr=PC); PC = PC + 1
    nn_msb = read_memory(addr=PC); PC = PC + 1
    nn = unsigned_16(lsb=nn_lsb, msb=nn_msb)
    BC = nn
```

Detailed timing and pseudocode



```
# M2
if IR == 0x01: # example: LD BC, nn
    Z = read_memory(addr=PC); PC = PC + 1
# M3
    W = read_memory(addr=PC); PC = PC + 1
# M4/M1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; BC = WZ
```

LD (nn), SP: Load from stack pointer (direct)

Load to the absolute address specified by the 16-bit operand nn, data from the 16-bit SP register.

Opcode 0b00001000/0x08

Duration 5 machine cycles

Length 3 bytes: opcode + LSB(nn) + MSB(nn) **Flags** -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x08:
```

```
    nn_lsb = read_memory(addr=PC); PC = PC + 1
```

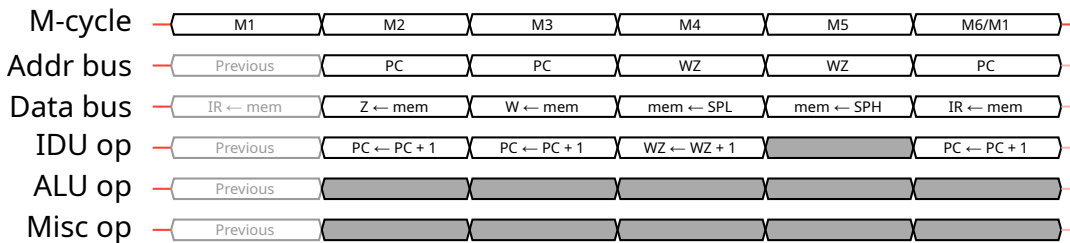
```
    nn_msb = read_memory(addr=PC); PC = PC + 1
```

```
    nn = unsigned_16(lsb=nn_lsb, msb=nn_msb)
```

```
    write_memory(addr=nn, data=lsb(SP)); nn = nn + 1
```

```
    write_memory(addr=nn, data=msb(SP))
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x08:
```

```
    Z = read_memory(addr=PC); PC = PC + 1
```

```
# M3
```

```
    W = read_memory(addr=PC); PC = PC + 1
```

```
# M4
```

```
    write_memory(addr=WZ, data=lsb(SP)); WZ = WZ + 1
```

```
# M5
```

```
    write_memory(addr=WZ, data=msb(SP))
```

```
# M6/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

LD SP, HL: Load stack pointer from HL

Load to the 16-bit SP register, data from the 16-bit HL register.

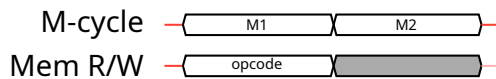
Opcode 0b111111001/0xF9

Duration 2 machine cycles

Length 1 byte: opcode

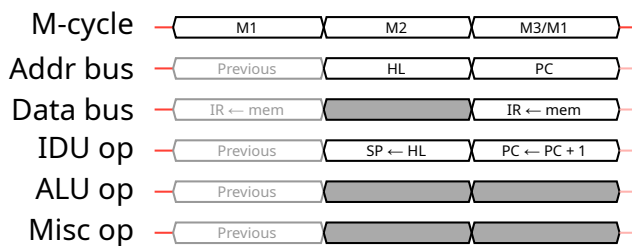
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1  
if opcode == 0xF9:  
    SP = HL
```

Detailed timing and pseudocode



```
# M2  
if IR == 0xF9:  
    SP = HL  
# M3/M1  
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

PUSH rr: Push to stack

Push to the stack memory, data from the 16-bit register `rr`.

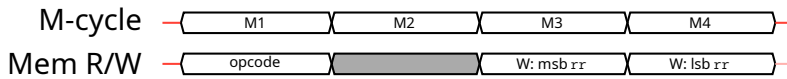
Opcode `0b11xx0101`/various

Duration 4 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

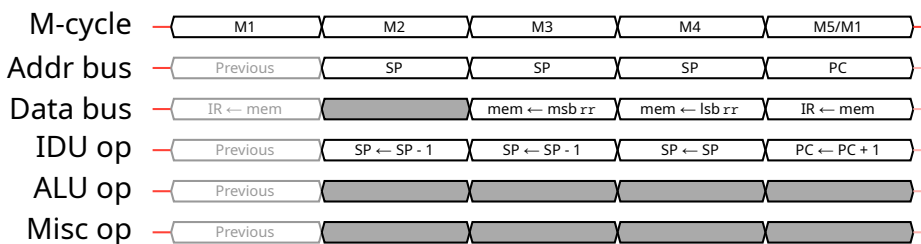
```
if opcode == 0xC5: # example: PUSH BC
```

```
    SP = SP - 1
```

```
    write_memory(addr=SP, data=msb(BC)); SP = SP - 1
```

```
    write_memory(addr=SP, data=lsb(BC))
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0xC5: # example: PUSH BC
```

```
    SP = SP - 1
```

```
# M3
```

```
    write_memory(addr=SP, data=msb(BC)); SP = SP - 1
```

```
# M4
```

```
    write_memory(addr=SP, data=lsb(BC))
```

```
# M5/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

POP rr: Pop from stack

Pops to the 16-bit register *rr*, data from the stack memory.

This instruction does not do calculations that affect flags, but POP AF completely replaces the F register value, so all flags are changed based on the 8-bit data that is read from memory.

Opcode 0b11xx0001/ various

Duration 3 machine cycles

Length 1 byte: opcode

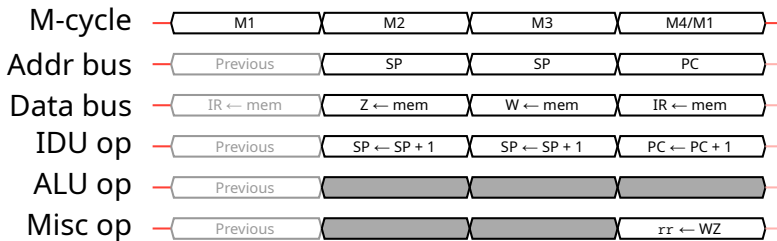
Flags See the instruction description

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xC1: # example: POP BC
    lsb = read_memory(addr=SP); SP = SP + 1
    msb = read_memory(addr=SP); SP = SP + 1
    BC = unsigned_16(lsb=lsb, msb=msb)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xC1: # example: POP BC
    Z = read_memory(addr=SP); SP = SP + 1
# M3
    W = read_memory(addr=SP); SP = SP + 1
# M4/M1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; BC = WZ
```

LD HL, SP+e: Load HL from adjusted stack pointer

Load to the HL register, 16-bit data calculated by adding the signed 8-bit operand *e* to the 16-bit value of the SP register.

Opcode 0b11111000/0xF8

Duration 3 machine cycles

Length 2 bytes: opcode + *e*

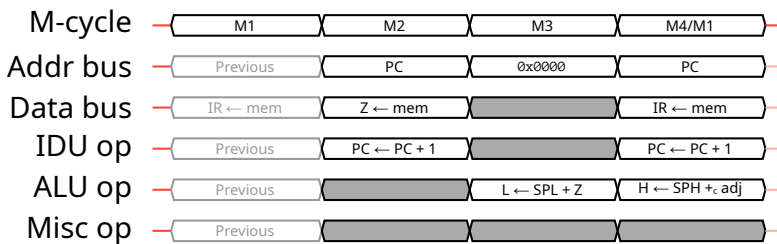
Flags Z = 0, N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xF8:
    e = signed_8(read_memory(addr=PC)); PC = PC + 1
    result, carry_per_bit = SP + e
    HL = result
    flags.Z = 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xF8:
    Z = read_memory(addr=PC); PC = PC + 1
# M3
result, carry_per_bit = lsb(SP) + Z
L = result
flags.Z = 0
flags.N = 0
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
Z_sign = bit(7, Z)
# M4/M1
adj = 0xFF if Z_sign else 0x00
result = msb(SP) + adj + flags.C
H = result
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

6.4 8-bit arithmetic and logical instructions

ADD r: Add (register)

Adds to the 8-bit A register, the 8-bit register r , and stores the result back into the A register.

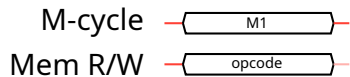
Opcode 0b10000xxx/various

Duration 1 machine cycle

Length 1 byte: opcode

Flags Z = ★, N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x80: # example: ADD B
```

```
    result, carry_per_bit = A + B
```

```
    A = result
```

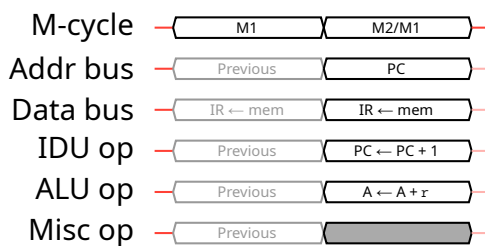
```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

```
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2/M1
```

```
if IR == 0x80: # example: ADD B
```

```
    result, carry_per_bit = A + B
```

```
    A = result
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

```
    flags.C = 1 if carry_per_bit[7] else 0
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


ADD (HL): Add (indirect HL)

Adds to the 8-bit A register, data from the absolute address specified by the 16-bit register HL, and stores the result back into the A register.

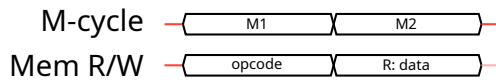
Opcode 0b10000110/0x86

Duration 2 machine cycles

Length 1 byte: opcode

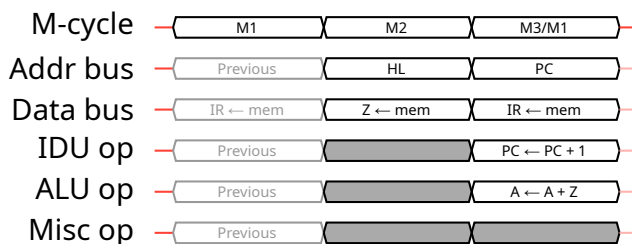
Flags Z = ★, N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x86:
    data = read_memory(addr=HL)
    result, carry_per_bit = A + data
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0x86:
    Z = read_memory(addr=HL)
# M3/M1
result, carry_per_bit = A + Z
A = result
flags.Z = 1 if result == 0 else 0
flags.N = 0
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

ADD n: Add (immediate)

Adds to the 8-bit A register, the immediate data n , and stores the result back into the A register.

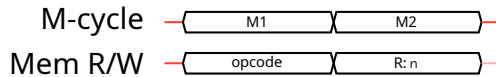
Opcode 0b11000110/0xC6

Duration 2 machine cycles

Length 2 bytes: opcode + n

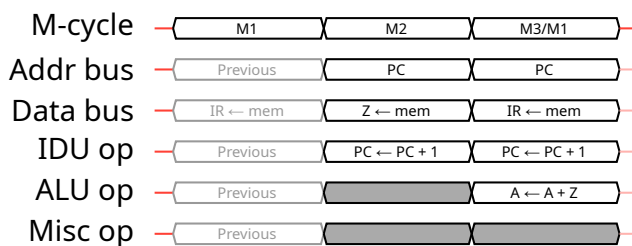
Flags Z = ★, N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xC6:
    n = read_memory(addr=PC); PC = PC + 1
    result, carry_per_bit = A + n
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xC6:
    Z = read_memory(addr=PC); PC = PC + 1
    # M3/M1
    result, carry_per_bit = A + Z
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

ADC r: Add with carry (register)

Adds to the 8-bit A register, the carry flag and the 8-bit register r, and stores the result back into the A register.

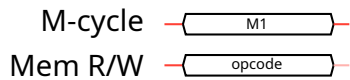
Opcode 0b10001xxx/variou

Length 1 byte: opcode

Duration 1 machine cycle

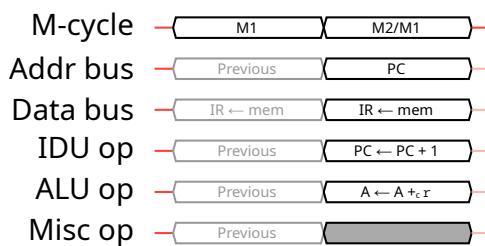
Flags Z = ★, N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x88: # example: ADC B
    result, carry_per_bit = A + B + flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2/M1
if IR == 0x88: # example: ADC B
    result, carry_per_bit = A + B + flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

ADC (HL): Add with carry (indirect HL)

Adds to the 8-bit A register, the carry flag and data from the absolute address specified by the 16-bit register HL, and stores the result back into the A register.

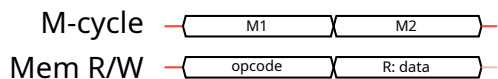
Opcode 0b10001110/0x8E

Duration 2 machine cycles

Length 1 byte: opcode

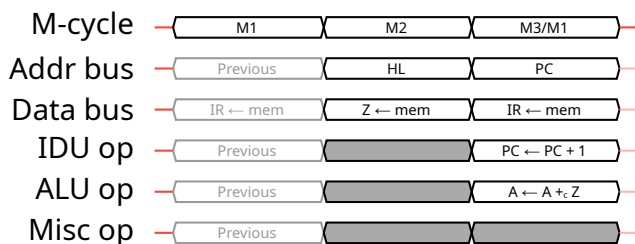
Flags Z = ★, N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x8E:
    data = read_memory(addr=HL)
    result, carry_per_bit = A + data + flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0x8E:
    Z = read_memory(addr=HL)
# M3/M1
result, carry_per_bit = A + Z + flags.C
A = result
flags.Z = 1 if result == 0 else 0
flags.N = 0
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

ADC n: Add with carry (immediate)

Adds to the 8-bit A register, the carry flag and the immediate data n, and stores the result back into the A register.

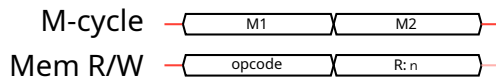
Opcode 0b11001110/0xCE

Duration 2 machine cycles

Length 2 bytes: opcode + n

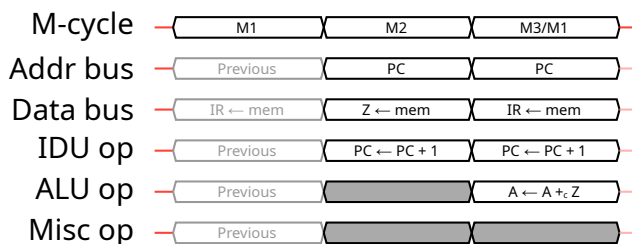
Flags Z = ★, N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCE:
    n = read_memory(addr=PC); PC = PC + 1
    result, carry_per_bit = A + n + flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xCE:
    Z = read_memory(addr=PC); PC = PC + 1
    # M3/M1
    result, carry_per_bit = A + Z + flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SUB r: Subtract (register)

Subtracts from the 8-bit A register, the 8-bit register r, and stores the result back into the A register.

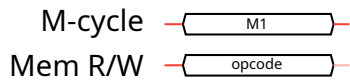
Opcode 0b10010xxx/variou

Length 1 byte: opcode

Duration 1 machine cycle

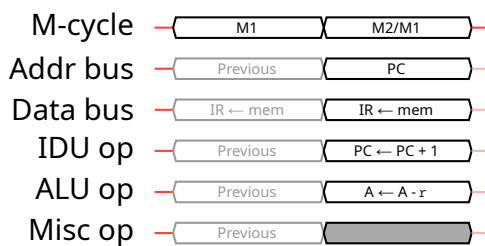
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x90: # example: SUB B
    result, carry_per_bit = A - B
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2/M1
if IR == 0x90: # example: SUB B
    result, carry_per_bit = A - B
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SUB (HL): Subtract (indirect HL)

Subtracts from the 8-bit A register, data from the absolute address specified by the 16-bit register HL, and stores the result back into the A register.

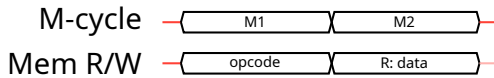
Opcode 0b10010110/0x96

Duration 2 machine cycles

Length 1 byte: opcode

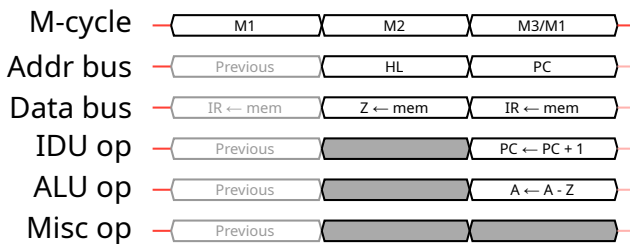
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x96:
    data = read_memory(addr=HL)
    result, carry_per_bit = A - data
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0x96:
    Z = read_memory(addr=HL)
# M3/M1
result, carry_per_bit = A - Z
A = result
flags.Z = 1 if result == 0 else 0
flags.N = 1
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SUB n: Subtract (immediate)

Subtracts from the 8-bit A register, the immediate data n, and stores the result back into the A register.

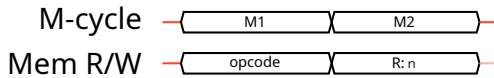
Opcode 0b11010110/0xD6

Duration 2 machine cycles

Length 2 bytes: opcode + n

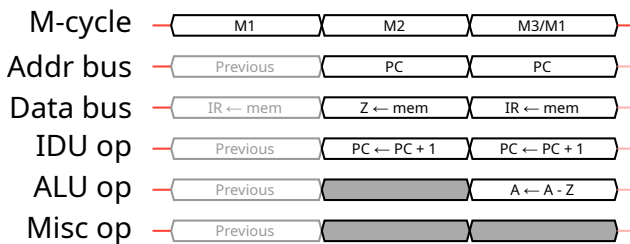
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xD6:
    n = read_memory(addr=PC); PC = PC + 1
    result, carry_per_bit = A - n
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xD6:
    Z = read_memory(addr=PC); PC = PC + 1
# M3/M1
result, carry_per_bit = A - Z
A = result
flags.Z = 1 if result == 0 else 0
flags.N = 1
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


SBC r: Subtract with carry (register)

Subtracts from the 8-bit A register, the carry flag and the 8-bit register r, and stores the result back into the A register.

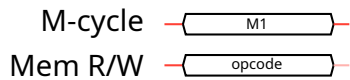
Opcode 0b10011xxx/variou

Length 1 byte: opcode

Duration 1 machine cycle

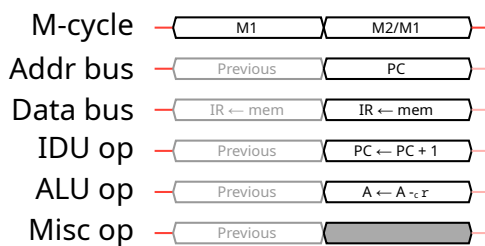
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x98: # example: SBC B
    result, carry_per_bit = A - B - flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2/M1
if IR == 0x98: # example: SBC B
    result, carry_per_bit = A - B - flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SBC (HL): Subtract with carry (indirect HL)

Subtracts from the 8-bit A register, the carry flag and data from the absolute address specified by the 16-bit register HL, and stores the result back into the A register.

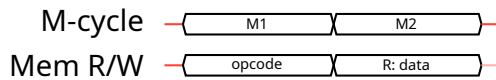
Opcode 0b10011110/0x9E

Duration 2 machine cycles

Length 1 byte: opcode

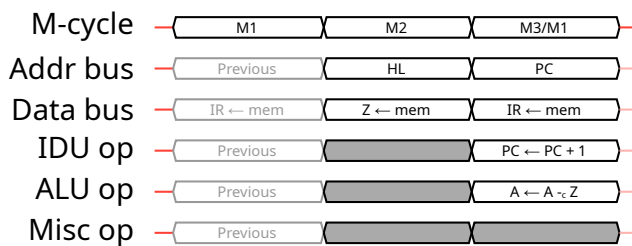
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x9E:
    data = read_memory(addr=HL)
    result, carry_per_bit = A - data - flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0x9E:
    Z = read_memory(addr=HL)
# M3/M1
result, carry_per_bit = A - Z - flags.C
A = result
flags.Z = 1 if result == 0 else 0
flags.N = 1
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SBC n: Subtract with carry (immediate)

Subtracts from the 8-bit A register, the carry flag and the immediate data n, and stores the result back into the A register.

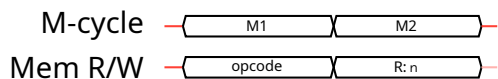
Opcode 0b11011110/0xDE

Duration 2 machine cycles

Length 2 bytes: opcode + n

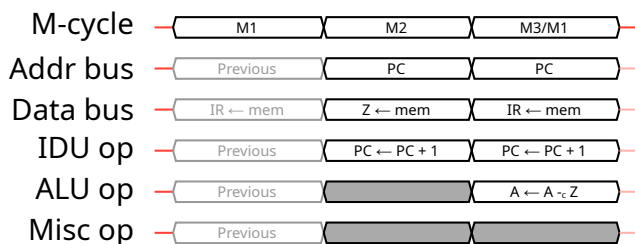
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xDE:
    n = read_memory(addr=PC); PC = PC + 1
    result, carry_per_bit = A - n - flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xDE:
    Z = read_memory(addr=PC); PC = PC + 1
    # M3/M1
    result, carry_per_bit = A - Z - flags.C
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

CP r: Compare (register)

Subtracts from the 8-bit A register, the 8-bit register *r*, and updates flags based on the result. This instruction is basically identical to SUB *r*, but does not update the A register.

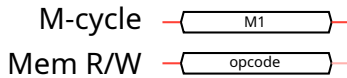
Opcode 0b10111xxx/variou

Duration 1 machine cycle

Length 1 byte: opcode

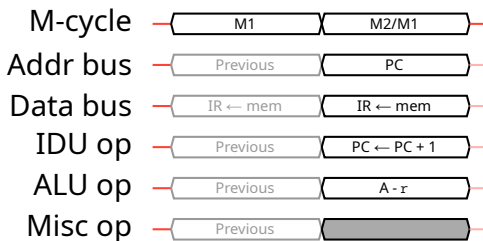
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xB8: # example: CP B
    result, carry_per_bit = A - B
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2/M1
if IR == 0xB8: # example: CP B
    result, carry_per_bit = A - B
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

CP (HL): Compare (indirect HL)

Subtracts from the 8-bit A register, data from the absolute address specified by the 16-bit register HL, and updates flags based on the result. This instruction is basically identical to SUB (HL), but does not update the A register.

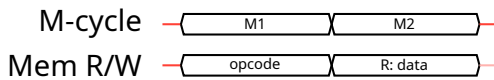
Opcode 0b10111110/0xBE

Length 1 byte: opcode

Duration 2 machine cycles

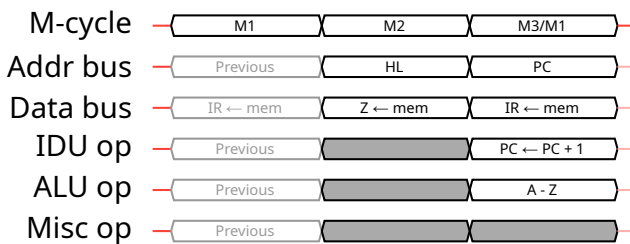
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xBE:
    data = read_memory(addr=HL)
    result, carry_per_bit = A - data
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xBE:
    Z = read_memory(addr=HL)
# M3/M1
result, carry_per_bit = A - Z
flags.Z = 1 if result == 0 else 0
flags.N = 1
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

CP n: Compare (immediate)

Subtracts from the 8-bit A register, the immediate data *n*, and updates flags based on the result. This instruction is basically identical to SUB *n*, but does not update the A register.

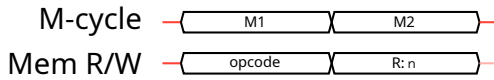
Opcode 0b11111110/0xFE

Duration 2 machine cycles

Length 2 bytes: opcode + *n*

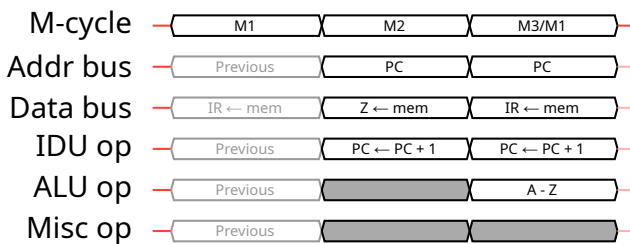
Flags Z = ★, N = 1, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xFE:
    n = read_memory(addr=PC); PC = PC + 1
    result, carry_per_bit = A - n
    flags.Z = 1 if result == 0 else 0
    flags.N = 1
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xFE:
    Z = read_memory(addr=PC); PC = PC + 1
# M3/M1
result, carry_per_bit = A - Z
flags.Z = 1 if result == 0 else 0
flags.N = 1
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
```

INC r: Increment (register)

Increments data in the 8-bit register r.

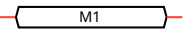
Opcode 0b00xxx100/variou

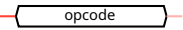
Length 1 byte: opcode

Duration 1 machine cycle

Flags Z = ★, N = 0, H = ★

Simple timing and pseudocode

M-cycle 

Mem R/W 

```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x04: # example: INC B
```

```
    result, carry_per_bit = B + 1
```

```
    B = result
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

Detailed timing and pseudocode

M-cycle 

Addr bus 

Data bus 

IDU op 

ALU op 

Misc op 

M2/M1

```
if IR == 0x04: # example: INC B
```

```
    result, carry_per_bit = B + 1
```

```
    B = result
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

INC (HL): Increment (indirect HL)

Increments data at the absolute address specified by the 16-bit register HL.

Opcode 0b00110100/0x34

Duration 3 machine cycles

Length 1 byte: opcode

Flags Z = ★, N = 0, H = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x34:
```

```
    data = read_memory(addr=HL)
```

```
    result, carry_per_bit = data + 1
```

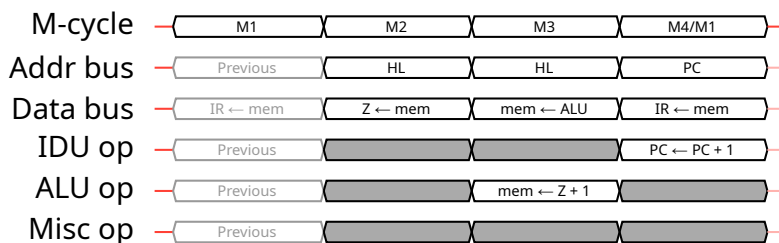
```
    write_memory(addr=HL, data=result)
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x34:
```

```
    Z = read_memory(addr=HL)
```

```
# M3
```

```
    result, carry_per_bit = Z + 1
```

```
    write_memory(addr=HL, data=result)
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

```
# M4/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


DEC r: Decrement (register)

Decrements data in the 8-bit register r.

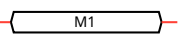
Opcode 0b00xxx101/variou

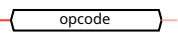
Length 1 byte: opcode

Duration 1 machine cycle

Flags Z = ★, N = 1, H = ★

Simple timing and pseudocode

M-cycle 

Mem R/W 

```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
# example: DEC B
```

```
if opcode == 0x05:
```

```
    result, carry_per_bit = B - 1
```

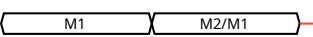
```
    B = result
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 1
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

Detailed timing and pseudocode

M-cycle 

Addr bus 

Data bus 

IDU op 

ALU op 

Misc op 

```
# M2/M1
```

```
if IR == 0x05: # example: DEC B
```

```
    result, carry_per_bit = B - 1
```

```
    B = result
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 1
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

DEC (HL): Decrement (indirect HL)

Decrements data at the absolute address specified by the 16-bit register HL.

Opcode 0b00110101/0x35

Duration 3 machine cycles

Length 1 byte: opcode

Flags Z = ★, N = 1, H = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x35:
```

```
    data = read_memory(addr=HL)
```

```
    result, carry_per_bit = data - 1
```

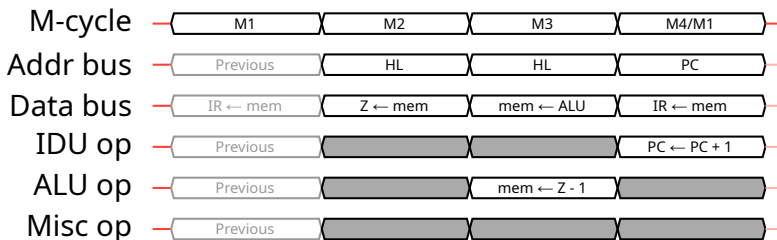
```
    write_memory(addr=HL, data=result)
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 1
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x35:
```

```
    Z = read_memory(addr=HL)
```

```
# M3
```

```
    result, carry_per_bit = Z - 1
```

```
    write_memory(addr=HL, data=result)
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 1
```

```
    flags.H = 1 if carry_per_bit[3] else 0
```

```
# M4/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

AND r: Bitwise AND (register)

Performs a bitwise AND operation between the 8-bit A register and the 8-bit register r, and stores the result back into the A register.

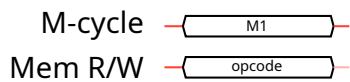
Opcode 0b10100xxx/variou

Length 1 byte: opcode

Duration 1 machine cycle

Flags Z = ★, N = 0, H = 1, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0xA0: # example: AND B
```

```
    result = A & B
```

```
    A = result
```

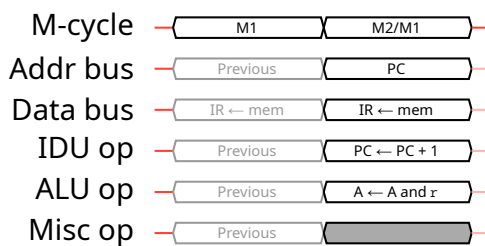
```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 1
```

```
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2/M1
```

```
if IR == 0xA0: # example: AND B
```

```
    result = A & B
```

```
    A = result
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 1
```

```
    flags.C = 0
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

AND (HL): Bitwise AND (indirect HL)

Performs a bitwise AND operation between the 8-bit A register and data from the absolute address specified by the 16-bit register HL, and stores the result back into the A register.

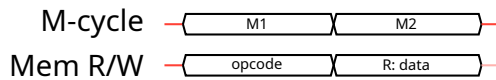
Opcode 0b10100110/0xA6

Duration 2 machine cycles

Length 1 byte: opcode

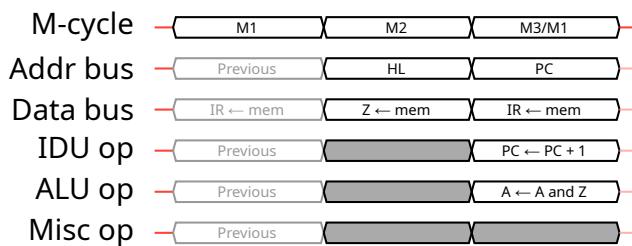
Flags Z = ★, N = 0, H = 1, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xA6:
    data = read_memory(addr=HL)
    result = A & data
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xA6:
    Z = read_memory(addr=HL)
    # M3/M1
    result = A & Z
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1
    flags.C = 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

AND n: Bitwise AND (immediate)

Performs a bitwise AND operation between the 8-bit A register and immediate data n, and stores the result back into the A register.

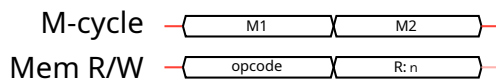
Opcode 0b11100110/0xE6

Length 2 bytes: opcode + n

Duration 2 machine cycles

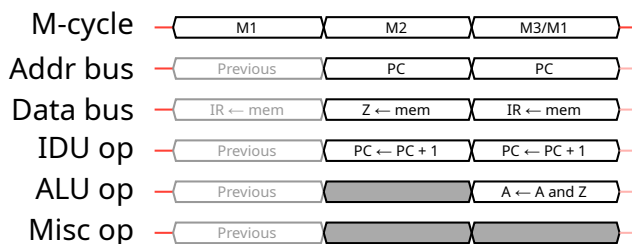
Flags Z = ★, N = 0, H = 1, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xE6:
    n = read_memory(addr=PC); PC = PC + 1
    result = A & n
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 1
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xE6:
    Z = read_memory(addr=PC); PC = PC + 1
# M3/M1
result = A & Z
A = result
flags.Z = 1 if result == 0 else 0
flags.N = 0
flags.H = 1
flags.C = 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

OR r: Bitwise OR (register)

Performs a bitwise OR operation between the 8-bit A register and the 8-bit register r , and stores the result back into the A register.

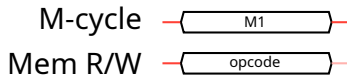
Opcode 0b10110xxx/ various

Length 1 byte: opcode

Duration 1 machine cycle

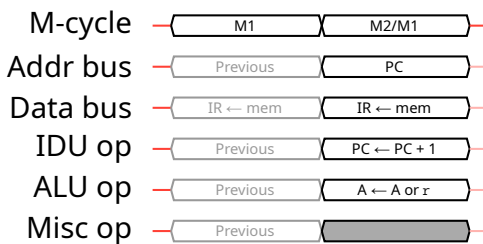
Flags Z = ★, N = 0, H = 0, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xB0: # example: OR B
    result = A | B
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2/M1
if IR == 0xB0: # example: OR B
    result = A | B
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

OR (HL): Bitwise OR (indirect HL)

Performs a bitwise OR operation between the 8-bit A register and data from the absolute address specified by the 16-bit register HL, and stores the result back into the A register.

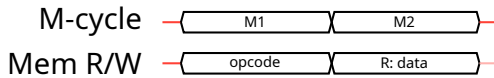
Opcode 0b10110110/0xB6

Duration 2 machine cycles

Length 1 byte: opcode

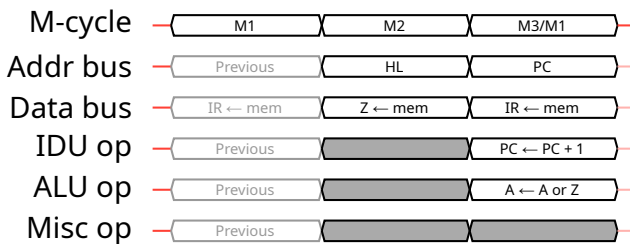
Flags Z = ★, N = 0, H = 0, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xB6:
    data = read_memory(addr=HL)
    result = A | data
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xB6:
    Z = read_memory(addr=HL)
    # M3/M1
    result = A | Z
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

OR n: Bitwise OR (immediate)

Performs a bitwise OR operation between the 8-bit A register and immediate data n , and stores the result back into the A register.

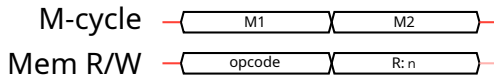
Opcode 0b11110110/0xF6

Length 2 bytes: opcode + n

Duration 2 machine cycles

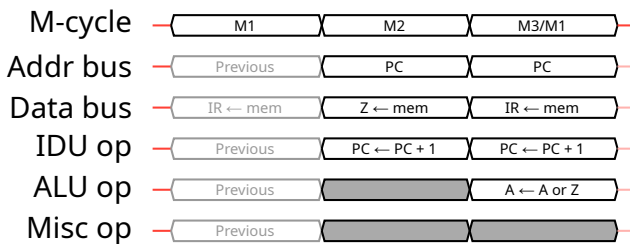
Flags Z = ★, N = 0, H = 0, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xF6:
    n = read_memory(addr=PC); PC = PC + 1
    result = A | n
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xF6:
    Z = read_memory(addr=PC); PC = PC + 1
# M3/M1
result = A | Z
A = result
flags.Z = 1 if result == 0 else 0
flags.N = 0
flags.H = 0
flags.C = 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


XOR r: Bitwise XOR (register)

Performs a bitwise XOR operation between the 8-bit A register and the 8-bit register r, and stores the result back into the A register.

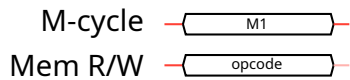
Opcode 0b10101xxx/varioux

Length 1 byte: opcode

Duration 1 machine cycle

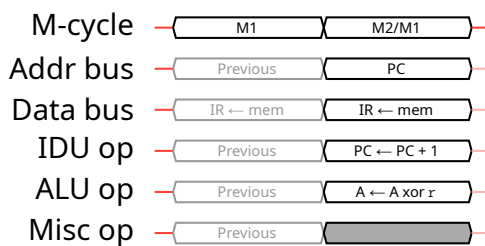
Flags Z = ★, N = 0, H = 0, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xA8: # example: XOR B
    result = A ^ B
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2/M1
if IR == 0xA8: # example: XOR B
    result = A ^ B
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

XOR (HL): Bitwise XOR (indirect HL)

Performs a bitwise XOR operation between the 8-bit A register and data from the absolute address specified by the 16-bit register HL, and stores the result back into the A register.

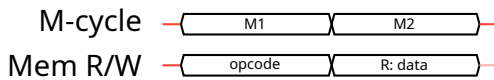
Opcode 0b10101110/0xAE

Length 1 byte: opcode

Duration 2 machine cycles

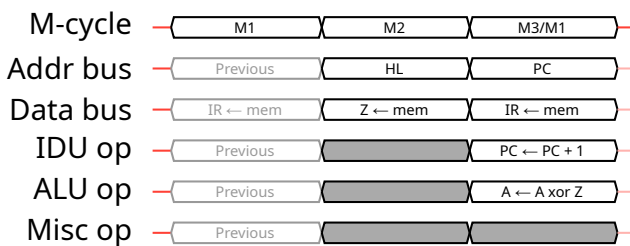
Flags Z = ★, N = 0, H = 0, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xAE:
    data = read_memory(addr=HL)
    result = A ^ data
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xAE:
    Z = read_memory(addr=HL)
    # M3/M1
    result = A ^ Z
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

XOR n: Bitwise XOR (immediate)

Performs a bitwise XOR operation between the 8-bit A register and immediate data n , and stores the result back into the A register.

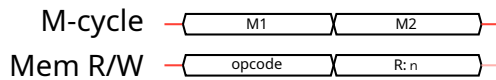
Opcode 0b11101110/0xEE

Length 2 bytes: opcode + n

Duration 2 machine cycles

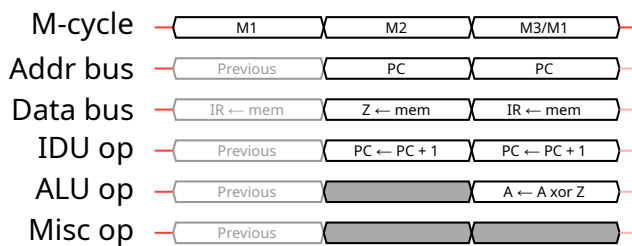
Flags Z = ★, N = 0, H = 0, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xEE:
    n = read_memory(addr=PC); PC = PC + 1
    result = A ^ n
    A = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0xEE:
    Z = read_memory(addr=PC); PC = PC + 1
# M3/M1
result = A ^ Z
A = result
flags.Z = 1 if result == 0 else 0
flags.N = 0
flags.H = 0
flags.C = 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

CCF: Complement carry flag

Flips the carry flag, and clears the N and H flags.

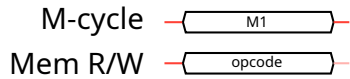
Opcode 0b00111111/0x3F

Length 1 byte: opcode

Duration 1 machine cycle

Flags N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

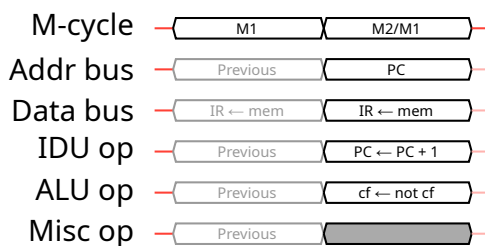
```
if opcode == 0x3F:
```

```
    flags.N = 0
```

```
    flags.H = 0
```

```
    flags.C = ~flags.C
```

Detailed timing and pseudocode



```
# M2/M1
```

```
if IR == 0x3F:
```

```
    flags.N = 0
```

```
    flags.H = 0
```

```
    flags.C = ~flags.C
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SCF: Set carry flag

Sets the carry flag, and clears the N and H flags.

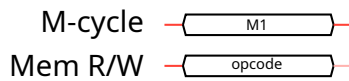
Opcode 0b00110111/0x37

Duration 1 machine cycle

Length 1 byte: opcode

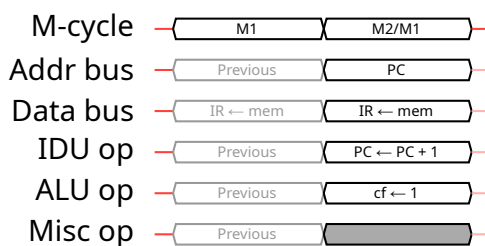
Flags N = 0, H = 0, C = 1

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x37:
    flags.N = 0
    flags.H = 0
    flags.C = 1
```

Detailed timing and pseudocode



```
# M2/M1
if IR == 0x37:
    flags.N = 0
    flags.H = 0
    flags.C = 1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

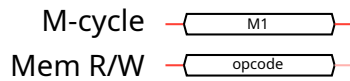
DAA: Decimal adjust accumulator

TODO

Opcode 0b00100111/0x27

Length 1 byte: opcode

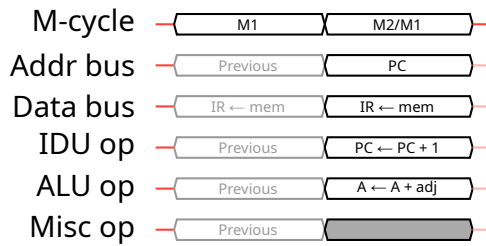
Simple timing and pseudocode



Duration 1 machine cycle

Flags Z = ★, H = 0, C = ★

Detailed timing and pseudocode



TODO

CPL: Complement accumulator

Flips all the bits in the 8-bit A register, and sets the N and H flags.

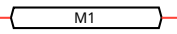
Opcode 0b00101111/0x2F

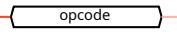
Duration 1 machine cycle

Length 1 byte: opcode

Flags N = 1, H = 1

Simple timing and pseudocode

M-cycle 

Mem R/W 

```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x2F:
```

```
    A = ~A
```

```
    flags.N = 1
```

```
    flags.H = 1
```

Detailed timing and pseudocode

M-cycle 

Addr bus 

Data bus 

IDU op 

ALU op 

Misc op 

```
# M2/M1
```

```
if IR == 0x2F:
```

```
    A = ~A
```

```
    flags.N = 1
```

```
    flags.H = 1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

6.5 16-bit arithmetic instructions

INC rr: Increment 16-bit register

Increments data in the 16-bit register rr.

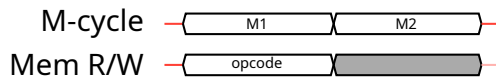
Opcode 0b00xx0011/variou

Length 1 byte: opcode

Duration 2 machine cycles

Flags -

Simple timing and pseudocode

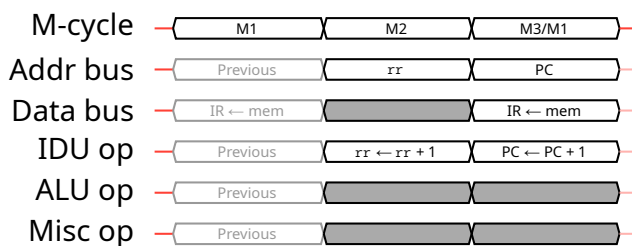


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x03: # example: INC BC
```

```
BC = BC + 1
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0x03: # example: INC BC
```

```
BC = BC + 1
```

```
# M3/M1
```

```
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


DEC rr: Decrement 16-bit register

Decrements data in the 16-bit register rr.

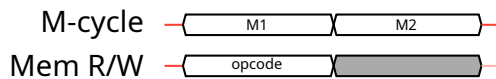
Opcode 0b00xx1011/variou

Duration 2 machine cycles

Length 1 byte: opcode

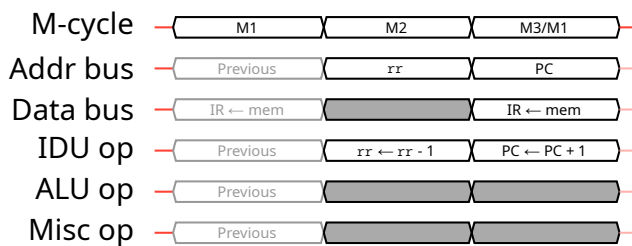
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1  
if opcode == 0x0B: # example: DEC BC  
    BC = BC - 1
```

Detailed timing and pseudocode



```
# M2  
if IR == 0x0B: # example: DEC BC  
    BC = BC - 1  
# M3/M1  
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

ADD HL, rr: Add (16-bit register)

Adds to the 16-bit HL register pair, the 16-bit register *rr*, and stores the result back into the HL register pair.

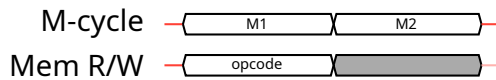
Opcode 0b00xx1001/variou

Length 1 byte: opcode

Duration 2 machine cycles

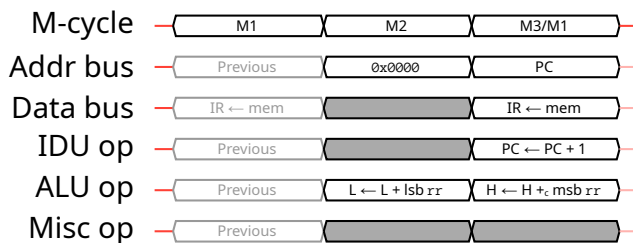
Flags N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x09: # example: ADD HL, BC
    result, carry_per_bit = HL + BC
    HL = result
    flags.N = 0
    flags.H = 1 if carry_per_bit[11] else 0
    flags.C = 1 if carry_per_bit[15] else 0
```

Detailed timing and pseudocode



```
# M2
if IR == 0x09: # example: ADD HL, BC
    result, carry_per_bit = L + C
    L = result
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
# M3/M1
result, carry_per_bit = H + B + flags.C
H = result
flags.N = 0
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

ADD SP, e: Add to stack pointer (relative)

Loads to the 16-bit SP register, 16-bit data calculated by adding the signed 8-bit operand *e* to the 16-bit value of the SP register.

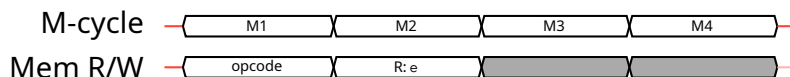
Opcode 0b11101000/0xE8

Duration 4 machine cycles

Length 2 bytes: opcode + *e*

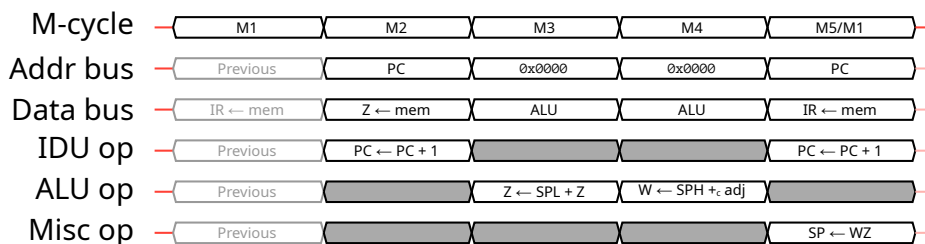
Flags Z = 0, N = 0, H = ★, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xE8:
    e = signed_8(read_memory(addr=PC)); PC = PC + 1
    result, carry_per_bit = SP + e
    SP = result
    flags.Z = 0
    flags.N = 0
    flags.H = 1 if carry_per_bit[3] else 0
    flags.C = 1 if carry_per_bit[7] else 0
```

Detailed timing and pseudocode



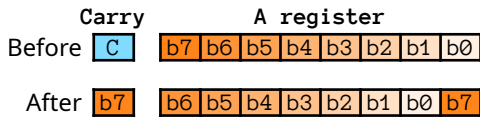
```
# M2
if IR == 0xE8:
    Z = read_memory(addr=PC); PC = PC + 1
# M3
result, carry_per_bit = lsb(SP) + Z
Z = result
flags.Z = 0
flags.N = 0
flags.H = 1 if carry_per_bit[3] else 0
flags.C = 1 if carry_per_bit[7] else 0
Z_sign = bit(7, Z)
# M4
adj = 0xFF if Z_sign else 0x00
result = msb(SP) + adj + flags.C
W = result
# M5/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1; SP = WZ
```


6.6 Rotate, shift, and bit operation instructions

RLCA: Rotate left circular (accumulator)

Rotates the 8-bit A register value left in a circular manner (carry flag is updated but not used).

Every bit is shifted to the left (e.g. bit 1 value is copied from bit 0). Bit 7 is copied both to bit 0 and the carry flag. Note that unlike the related RLC r instruction, RLCA always sets the zero flag to 0 without looking at the resulting value of the calculation.



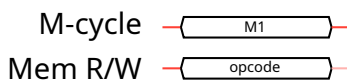
Opcode 0b00000111/0x07

Length 1 byte: opcode

Duration 1 machine cycle

Flags Z = 0, N = 0, H = 0, C = ★

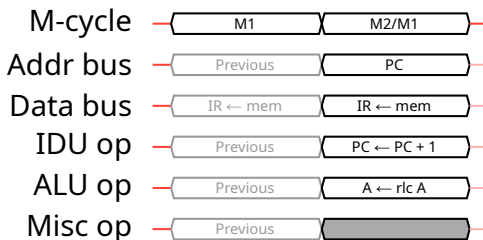
Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x07:  
    b7 = A[7]  
    A = from_bits(7..1=A[6..0], 0=b7)  
    flags.Z = 0  
    flags.N = 0  
    flags.H = 0  
    flags.C = 1 if b7 else 0
```

Detailed timing and pseudocode

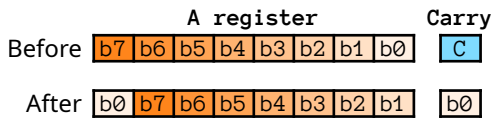


```
# M2/M1  
if IR == 0x07:  
    b7 = A[7]  
    result = from_bits(7..1=A[6..0], 0=b7)  
    A = result  
    flags.Z = 0  
    flags.N = 0  
    flags.H = 0  
    flags.C = 1 if b7 else 0  
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RRCA: Rotate right circular (accumulator)

Rotates the 8-bit A register value right in a circular manner (carry flag is updated but not used).

Every bit is shifted to the right (e.g. bit 1 value is copied to bit 0). Bit 0 is copied both to bit 7 and the carry flag. Note that unlike the related RRC r instruction, RRCA always sets the zero flag to 0 without looking at the resulting value of the calculation.



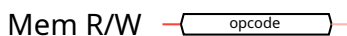
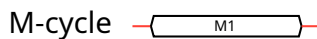
Opcode 0b00001111/0x0F

Length 1 byte: opcode

Duration 1 machine cycle

Flags Z = 0, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x0F:
```

```
    b0 = A[0]
```

```
    A = from_bits(7=b0, 6..0=A[7..1])
```

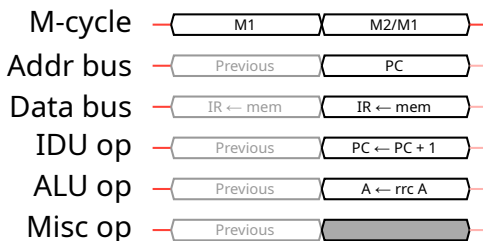
```
    flags.Z = 0
```

```
    flags.N = 0
```

```
    flags.H = 0
```

```
    flags.C = 1 if b0 else 0
```

Detailed timing and pseudocode



M2/M1

```
if IR == 0x0F:
```

```
    b0 = A[0]
```

```
    result = from_bits(7=b0, 6..0=A[7..1])
```

```
    A = result
```

```
    flags.Z = 0
```

```
    flags.N = 0
```

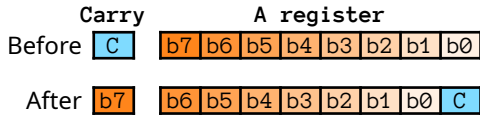
```
    flags.H = 0
```

```
    flags.C = 1 if b0 else 0
```

RLA: Rotate left (accumulator)

Rotates the 8-bit A register value left through the carry flag.

Every bit is shifted to the left (e.g. bit 1 value is copied from bit 0). The carry flag is copied to bit 0, and bit 7 is copied to the carry flag. Note that unlike the related RL r instruction, RLA always sets the zero flag to 0 without looking at the resulting value of the calculation.



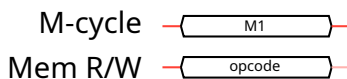
Opcode 0b00010111/0x17

Length 1 byte: opcode

Duration 1 machine cycle

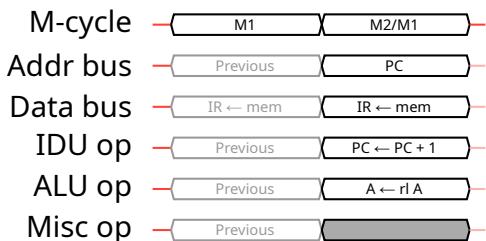
Flags Z = 0, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x17:
    b7 = A[7]
    A = from_bits(7..1=A[6..0], 0=flags.C)
    flags.Z = 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
```

Detailed timing and pseudocode

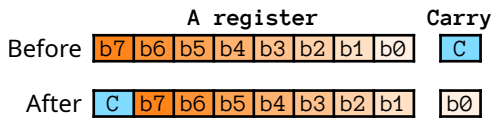


```
# M2/M1
if IR == 0x17:
    b7 = A[7]
    result = from_bits(7..1=A[6..0], 0=flags.C)
    A = result
    flags.Z = 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
```

RRA: Rotate right (accumulator)

Rotates the 8-bit A register value right through the carry flag.

Every bit is shifted to the right (e.g. bit 1 value is copied to bit 0). The carry flag is copied to bit 7, and bit 0 is copied to the carry flag. Note that unlike the related RR r instruction, RRA always sets the zero flag to 0 without looking at the resulting value of the calculation.



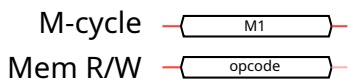
Opcode 0b00011111/0x1F

Length 1 byte: opcode

Duration 1 machine cycle

Flags Z = 0, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x1F:
```

```
    b0 = A[0]
```

```
    A = from_bits(7=flags.C, 6..0=A[7..1])
```

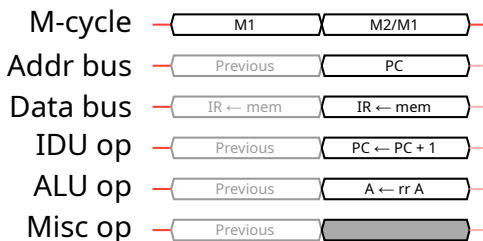
```
    flags.Z = 0
```

```
    flags.N = 0
```

```
    flags.H = 0
```

```
    flags.C = 1 if b0 else 0
```

Detailed timing and pseudocode



```
# M2/M1
```

```
if IR == 0x1F:
```

```
    b0 = A[0]
```

```
    result = from_bits(7=flags.C, 6..0=A[7..1])
```

```
    A = result
```

```
    flags.Z = 0
```

```
    flags.N = 0
```

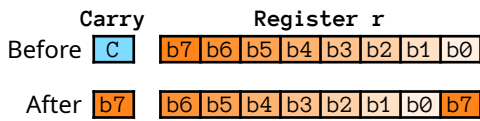
```
    flags.H = 0
```

```
    flags.C = 1 if b0 else 0
```


RLC r: Rotate left circular (register)

Rotates the 8-bit register r value left in a circular manner (carry flag is updated but not used).

Every bit is shifted to the left (e.g. bit 1 value is copied from bit 0). Bit 7 is copied both to bit 0 and the carry flag.



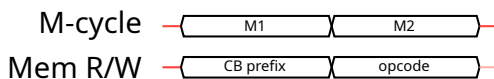
Opcode $0b00000xxx$ /various

Length 2 bytes: CB prefix + opcode

Duration 2 machine cycles

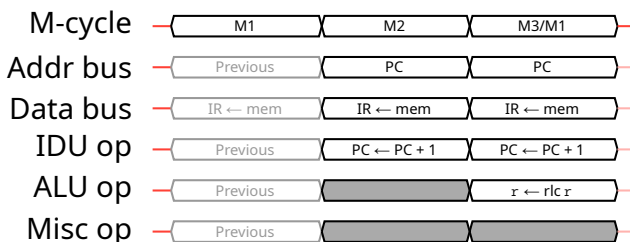
Flags $Z = \star, N = 0, H = 0, C = \star$

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x00: # example: RLC B
    b7 = B[7]
    B = from_bits(7..1=B[6..0], 0=b7)
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
```

Detailed timing and pseudocode

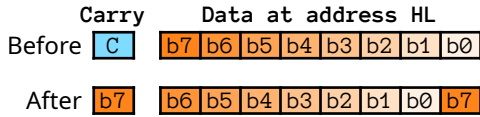


```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x00: # example: RLC B
    b7 = B[7]
    result = from_bits(7..1=B[6..0], 0=b7)
    B = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RLC (HL): Rotate left circular (indirect HL)

Rotates, the 8-bit data at the absolute address specified by the 16-bit register HL, left in a circular manner (carry flag is updated but not used).

Every bit is shifted to the left (e.g. bit 1 value is copied from bit 0). Bit 7 is copied both to bit 0 and the carry flag.



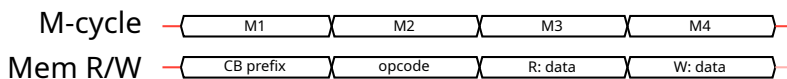
Opcode 0x06

Duration 4 machine cycles

Length 2 bytes: CB prefix + opcode

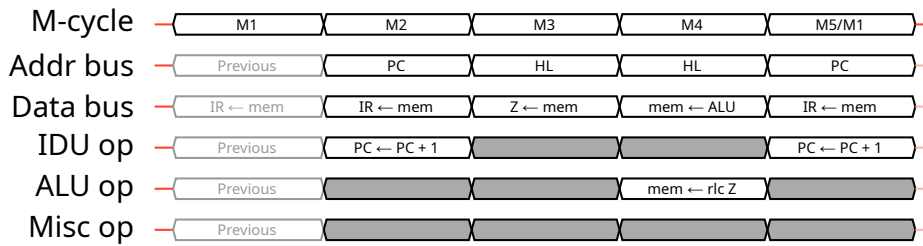
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x06:
    data = read_memory(addr=HL)
    b7 = data[7]
    result = from_bits(7..1=data[6..0], 0=b7)
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
    write_memory(addr=HL, data=result)
```

Detailed timing and pseudocode



M2

if IR == 0xCB:

cb_mode = 1

IR = fetch_cycle(addr=PC); PC = PC + 1

M3

if cb_mode and IR == 0x06:

Z = read_memory(addr=HL)

M4

b7 = Z[7]

result = from_bits(7..1=Z[6..0], 0=b7)

flags.Z = 1 if result == 0 else 0

flags.N = 0

flags.H = 0

flags.C = 1 if b7 else 0

write_memory(addr=HL, data=result)

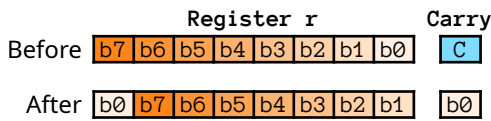
M5/M1

IR, intr = fetch_cycle(addr=PC); PC = PC + 1

RRC r: Rotate right circular (register)

Rotates the 8-bit register r value right in a circular manner (carry flag is updated but not used).

Every bit is shifted to the right (e.g. bit 1 value is copied to bit 0). Bit 0 is copied both to bit 7 and the carry flag.



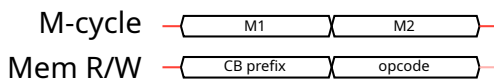
Opcode 0b00001xxx/variou

Duration 2 machine cycles

Length 2 bytes: CB prefix + opcode

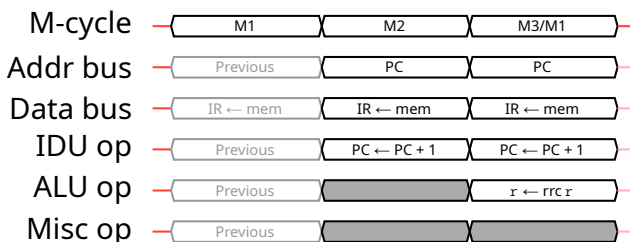
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x08: # example: RRC B
    b0 = B[0]
    B = from_bits(7..1=B[7..1], 0=b0)
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
```

Detailed timing and pseudocode

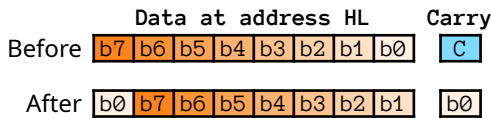


```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x08: # example: RRC B
    b0 = B[0]
    result = from_bits(7..1=B[7..1], 0=b0)
    B = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RRC (HL): Rotate right circular (indirect HL)

Rotates, the 8-bit data at the absolute address specified by the 16-bit register HL, right in a circular manner (carry flag is updated but not used).

Every bit is shifted to the right (e.g. bit 1 value is copied to bit 0). Bit 0 is copied both to bit 7 and the carry flag.



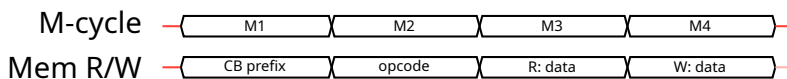
Opcode 0x0E

Length 2 bytes: CB prefix + opcode

Duration 4 machine cycles

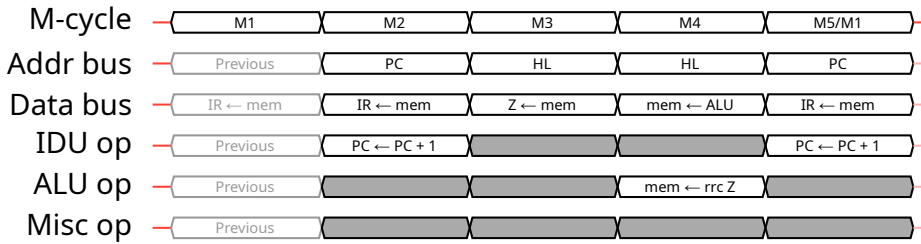
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x0E:
    data = read_memory(addr=HL)
    b0 = data[0]
    result = from_bits(7=b0, 6..0=data[7..1])
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
    write_memory(addr=HL, data=result)
```

Detailed timing and pseudocode



M2

if IR == 0xCB:

cb_mode = 1

IR = fetch_cycle(addr=PC); PC = PC + 1

M3

if cb_mode and IR == 0x0E:

Z = read_memory(addr=HL)

M4

b0 = Z[0]

result = from_bits(7=b0, 6..0=Z[7..1])

flags.Z = 1 if result == 0 else 0

flags.N = 0

flags.H = 0

flags.C = 1 if b0 else 0

write_memory(addr=HL, data=result)

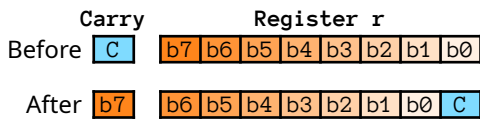
M5/M1

IR, intr = fetch_cycle(addr=PC); PC = PC + 1

RL r: Rotate left (register)

Rotates the 8-bit register r value left through the carry flag.

Every bit is shifted to the left (e.g. bit 1 value is copied from bit 0). The carry flag is copied to bit 0, and bit 7 is copied to the carry flag.



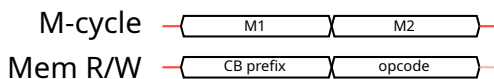
Opcode 0b00010xxx/variou

Length 2 bytes: CB prefix + opcode

Duration 2 machine cycles

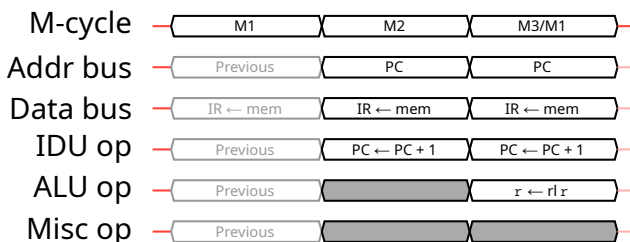
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x10: # example: RL B
    b7 = B[7]
    B = from_bits(7..1=B[6..0], 0=flags.C)
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
```

Detailed timing and pseudocode

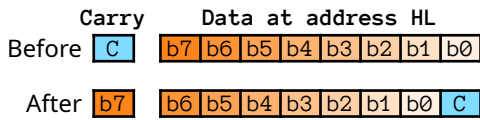


```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x10: # example: RL B
    b7 = B[7]
    result = from_bits(7..1=B[6..0], 0=flags.C)
    B = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RL (HL): Rotate left (indirect HL)

Rotates, the 8-bit data at the absolute address specified by the 16-bit register HL, left through the carry flag.

Every bit is shifted to the left (e.g. bit 1 value is copied from bit 0). The carry flag is copied to bit 0, and bit 7 is copied to the carry flag.



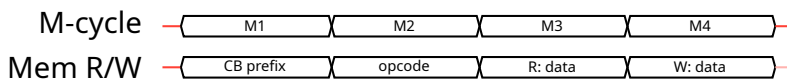
Opcode 0x16

Duration 4 machine cycles

Length 2 bytes: CB prefix + opcode

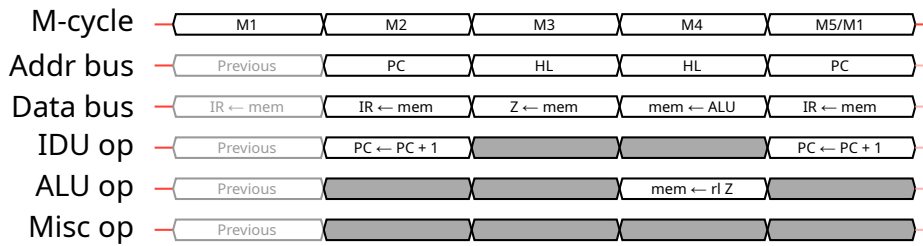
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x16:
    data = read_memory(addr=HL)
    b7 = data[7]
    result = from_bits(7..1=data[6..0], 0=flags.C)
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
    write_memory(addr=HL, data=result)
```


Detailed timing and pseudocode



M2

```
if IR == 0xCB:
```

```
    cb_mode = 1
```

```
    IR = fetch_cycle(addr=PC); PC = PC + 1
```

M3

```
if cb_mode and IR == 0x16:
```

```
    Z = read_memory(addr=HL)
```

M4

```
    b7 = Z[7]
```

```
    result = from_bits(7..1=Z[6..0], 0=flags.C)
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 0
```

```
    flags.C = 1 if b7 else 0
```

```
    write_memory(addr=HL, data=result)
```

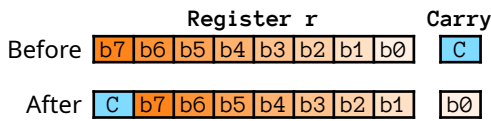
M5/M1

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RR r: Rotate right (register)

Rotates the 8-bit register *r* value right through the carry flag.

Every bit is shifted to the right (e.g. bit 1 value is copied to bit 0). The carry flag is copied to bit 7, and bit 0 is copied to the carry flag.



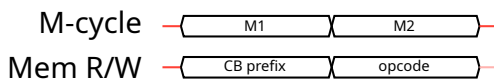
Opcode 0b00011xxx/variou

Duration 2 machine cycles

Length 2 bytes: CB prefix + opcode

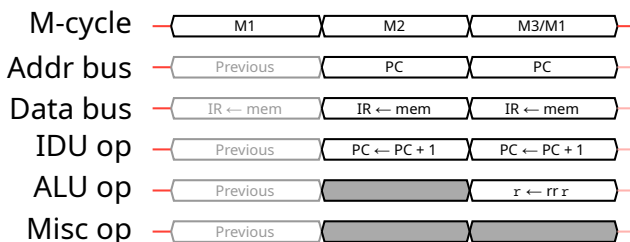
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x18: # example: RR B
    b0 = B[0]
    B = from_bits(7=flags.C, 6..0=B[7..1])
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
```

Detailed timing and pseudocode

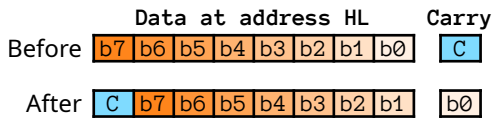


```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x18: # example: RR B
    b0 = B[0]
    result = from_bits(7=flags.C, 6..0=B[7..1])
    B = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RR (HL): Rotate right (indirect HL)

Rotates, the 8-bit data at the absolute address specified by the 16-bit register HL, right through the carry flag.

Every bit is shifted to the right (e.g. bit 1 value is copied to bit 0). The carry flag is copied to bit 7, and bit 0 is copied to the carry flag.



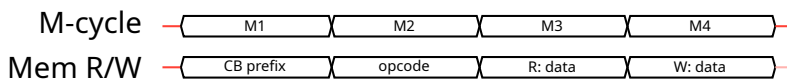
Opcode 0x1E

Length 2 bytes: CB prefix + opcode

Duration 4 machine cycles

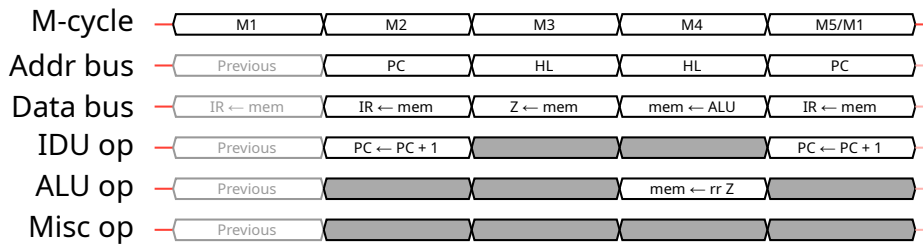
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x1E:
    data = read_memory(addr=HL)
    b0 = data[0]
    result = from_bits(7=flags.C, 6..0=data[7..1])
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
    write_memory(addr=HL, data=result)
```

Detailed timing and pseudocode



M2

```
if IR == 0xCB:
```

```
    cb_mode = 1
```

```
    IR = fetch_cycle(addr=PC); PC = PC + 1
```

M3

```
if cb_mode and IR == 0x1E:
```

```
    Z = read_memory(addr=HL)
```

M4

```
    b0 = Z[0]
```

```
    result = from_bits(7=flags.C, 6..0=Z[7..1])
```

```
    flags.Z = 1 if result == 0 else 0
```

```
    flags.N = 0
```

```
    flags.H = 0
```

```
    flags.C = 1 if b0 else 0
```

```
    write_memory(addr=HL, data=result)
```

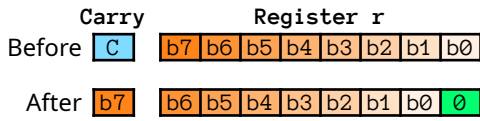
M5/M1

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SLA r: Shift left arithmetic (register)

Shifts the 8-bit register r value left by one bit using an arithmetic shift.

Bit 7 is shifted to the carry flag, and bit 0 is set to a fixed value of 0.



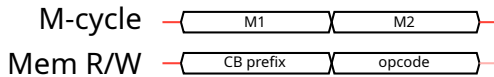
Opcode 0b00100xxx/various

Duration 2 machine cycles

Length 2 bytes: CB prefix + opcode

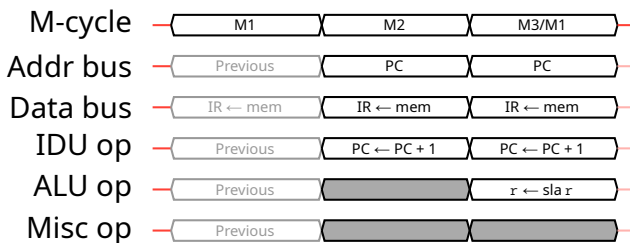
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x20: # example: SLA B
    b7 = B[7]
    B = from_bits(7..1=B[6..0], 0=0)
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
```

Detailed timing and pseudocode

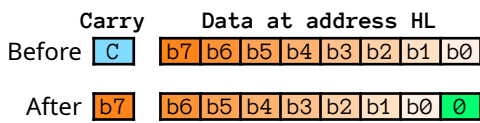


```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x20: # example: SLA B
    b7 = B[7]
    result = from_bits(7..1=B[6..0], 0=0)
    B = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SLA (HL): Shift left arithmetic (indirect HL)

Shifts, the 8-bit value at the address specified by the HL register, left by one bit using an arithmetic shift.

Bit 7 is shifted to the carry flag, and bit 0 is set to a fixed value of 0.



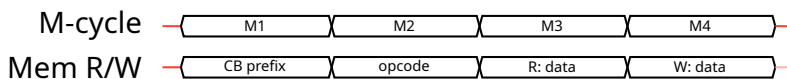
Opcode 0x26

Duration 4 machine cycles

Length 2 bytes: CB prefix + opcode

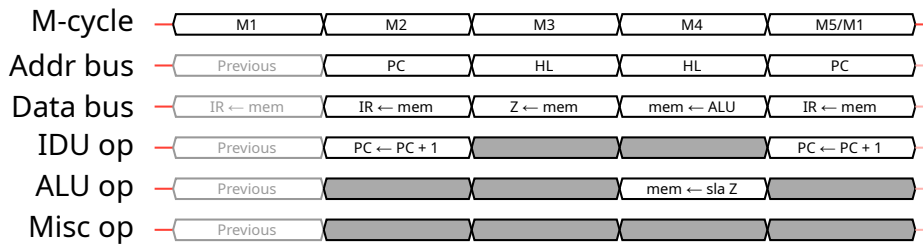
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x26:
    data = read_memory(addr=HL)
    b7 = data[7]
    result = from_bits(7..1=data[6..0], 0=0)
    data = result
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b7 else 0
```

Detailed timing and pseudocode



M2

if IR == 0xCB:

cb_mode = 1

IR = fetch_cycle(addr=PC); PC = PC + 1

M3

if cb_mode and IR == 0x26:

Z = read_memory(addr=HL)

M4

b7 = Z[7]

result = from_bits(7..1=Z[6..0], 0=0)

flags.Z = 1 if result == 0 else 0

flags.N = 0

flags.H = 0

flags.C = 1 if b7 else 0

write_memory(addr=HL, data=result)

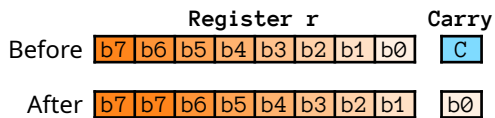
M5/M1

IR, intr = fetch_cycle(addr=PC); PC = PC + 1

SRA r: Shift right arithmetic (register)

Shifts the 8-bit register r value right by one bit using an arithmetic shift.

Bit 7 retains its value, and bit 0 is shifted to the carry flag.



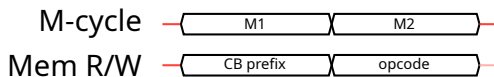
Opcode 0b00101xxx/varioux

Duration 2 machine cycles

Length 2 bytes: CB prefix + opcode

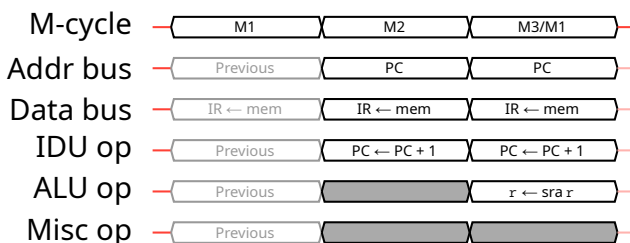
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x28: # example: SRA B
    b0 = B[0]
    B = from_bits(7=B[7], 6..0=B[7..1])
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
```

Detailed timing and pseudocode

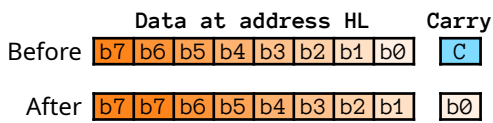


```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x28: # example: SRA B
    b0 = B[0]
    result = from_bits(7=B[7], 6..0=B[7..1])
    B = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


SRA (HL): Shift right arithmetic (indirect HL)

Shifts, the 8-bit value at the address specified by the HL register, right by one bit using an arithmetic shift.

Bit 7 retains its value, and bit 0 is shifted to the carry flag.



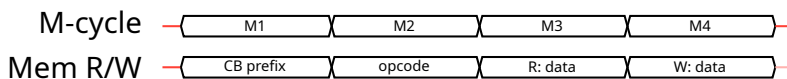
Opcode 0x2E

Duration 4 machine cycles

Length 2 bytes: CB prefix + opcode

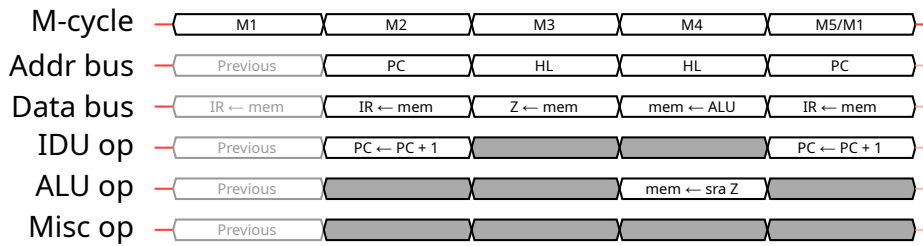
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x2E:
    data = read_memory(addr=HL)
    b0 = data[0]
    result = from_bits(7=0, 6..0=data[7..1])
    data = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
```

Detailed timing and pseudocode



M2

if IR == 0xCB:

cb_mode = 1

IR = fetch_cycle(addr=PC); PC = PC + 1

M3

if cb_mode and IR == 0x2E:

Z = read_memory(addr=HL)

M4

b0 = Z[0]

result = from_bits(7=0, 6..0=Z[7..1])

flags.Z = 1 if result == 0 else 0

flags.N = 0

flags.H = 0

flags.C = 1 if b0 else 0

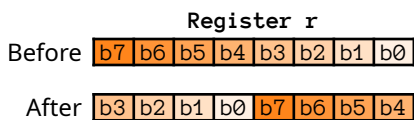
write_memory(addr=HL, data=result)

M5/M1

IR, intr = fetch_cycle(addr=PC); PC = PC + 1

SWAP r: Swap nibbles (register)

Swaps the high and low 4-bit nibbles of the 8-bit register r.



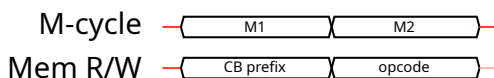
Opcode 0b00110xxx/variou

Length 2 bytes: CB prefix + opcode

Duration 2 machine cycles

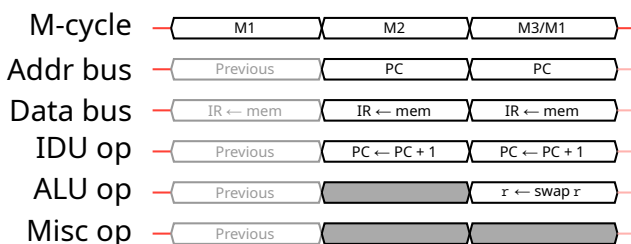
Flags Z = ★, N = 0, H = 0, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x30: # example: SWAP B
    B = from_bits(7..4=B[3..0], 3..0=B[7..4])
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
```

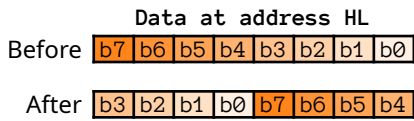
Detailed timing and pseudocode



```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x30: # example: SWAP B
    result = from_bits(7..4=B[3..0], 3..0=B[7..4])
    B = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SWAP (HL): Swap nibbles (indirect HL)

Swaps the high and low 4-bit nibbles of the 8-bit data at the absolute address specified by the 16-bit register HL.



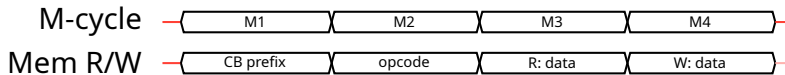
Opcode 0x36

Length 2 bytes: CB prefix + opcode

Duration 4 machine cycles

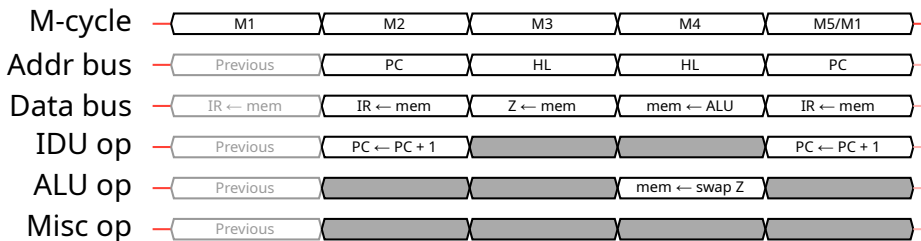
Flags Z = ★, N = 0, H = 0, C = 0

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
    if opcode == 0x36:
        data = read_memory(addr=HL)
        result = from_bits(7..4=data[3..0], 3..0=data[7..4])
        flags.Z = 1 if result == 0 else 0
        flags.N = 0
        flags.H = 0
        flags.C = 0
        write_memory(addr=HL, data=result)
```

Detailed timing and pseudocode

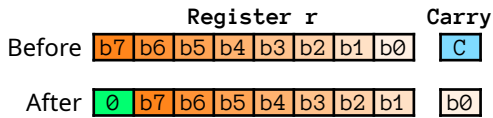


```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3
if cb_mode and IR == 0x36:
    Z = read_memory(addr=HL)
# M4
result = from_bits(7..4=Z[3..0], 3..0=Z[7..4])
flags.Z = 1 if result == 0 else 0
flags.N = 0
flags.H = 0
flags.C = 0
write_memory(addr=HL, data=result)
# M5/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SRL r: Shift right logical (register)

Shifts the 8-bit register r value right by one bit using a logical shift.

Bit 7 is set to a fixed value of 0, and bit 0 is shifted to the carry flag.



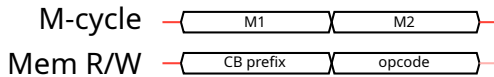
Opcode 0b00111xxx/various

Duration 2 machine cycles

Length 2 bytes: CB prefix + opcode

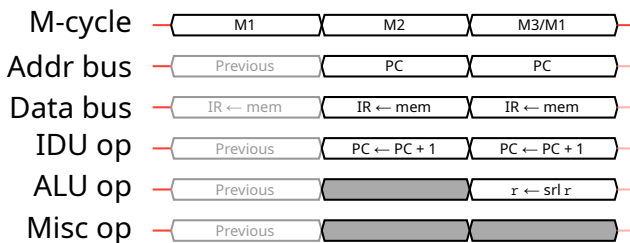
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x38: # example: SRL B
    b0 = B[0]
    B = from_bits(7=0, 6..0=B[7..1])
    flags.Z = 1 if B == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
```

Detailed timing and pseudocode

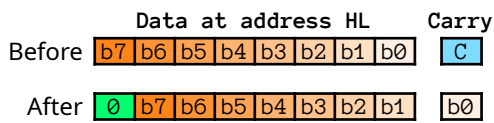


```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x38: # example: SRL B
    b0 = B[0]
    result = from_bits(7=0, 6..0=B[7..1])
    B = result
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SRL (HL): Shift right logical (indirect HL)

Shifts, the 8-bit value at the address specified by the HL register, right by one bit using a logical shift.

Bit 7 is set to a fixed value of 0, and bit 0 is shifted to the carry flag.



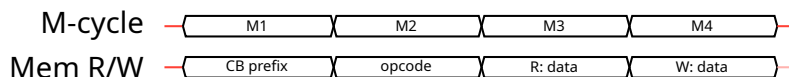
Opcode 0x3E

Duration 4 machine cycles

Length 2 bytes: CB prefix + opcode

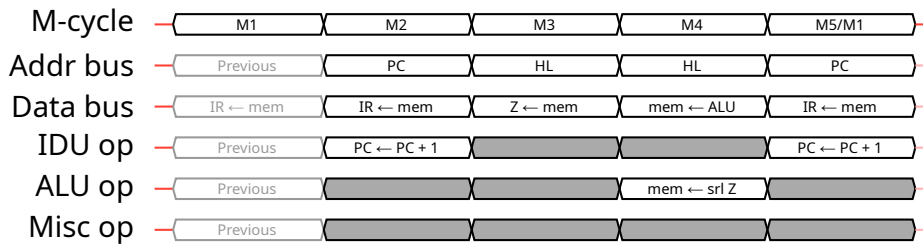
Flags Z = ★, N = 0, H = 0, C = ★

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x3E:
    data = read_memory(addr=HL)
    b0 = data[0]
    result = from_bits(7=0, 6..0=data[7..1])
    flags.Z = 1 if result == 0 else 0
    flags.N = 0
    flags.H = 0
    flags.C = 1 if b0 else 0
    write_memory(addr=HL, data=result)
```

Detailed timing and pseudocode



M2

if IR == 0xCB:

cb_mode = 1

IR = fetch_cycle(addr=PC); PC = PC + 1

M3

if cb_mode and IR == 0x3E:

Z = read_memory(addr=HL)

M4

b0 = Z[0]

result = from_bits(7=0, 6..0=Z[7..1])

flags.Z = 1 if result == 0 else 0

flags.N = 0

flags.H = 0

flags.C = 1 if b0 else 0

write_memory(addr=HL, data=result)

M5/M1

IR, intr = fetch_cycle(addr=PC); PC = PC + 1

BIT b, r: Test bit (register)

Tests the bit b of the 8-bit register r.

The zero flag is set to 1 if the chosen bit is 0, and 0 otherwise.

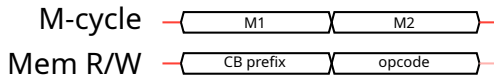
Opcode 0b01xxxxxx/various

Duration 2 machine cycles

Length 2 bytes: CB prefix + opcode

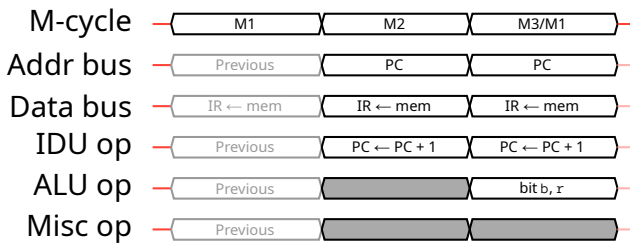
Flags Z = ★, N = 0, H = 1

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
    if opcode == 0x40: # example: BIT 0, B
        flags.Z = 1 if B[0] == 0 else 0
        flags.N = 0
        flags.H = 1
```

Detailed timing and pseudocode



```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x40: # example: BIT 0, B
    flags.Z = 1 if B[0] == 0 else 0
    flags.N = 0
    flags.H = 1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


BIT b, (HL): Test bit (indirect HL)

Tests the bit b of the 8-bit data at the absolute address specified by the 16-bit register HL.

The zero flag is set to 1 if the chosen bit is 0, and 0 otherwise.

Opcode 0b01xxx110/variou

Duration 3 machine cycles

Length 2 bytes: CB prefix + opcode

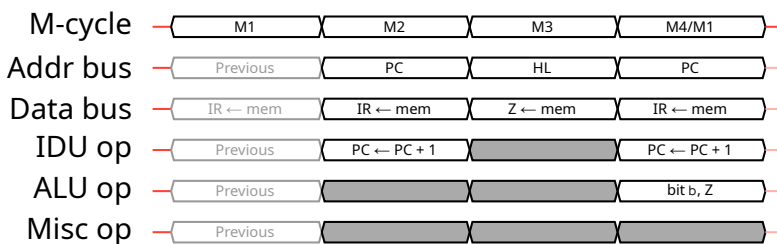
Flags Z = ★, N = 0, H = 1

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
    if opcode == 0x46: # example: BIT 0, (HL)
        data = read_memory(addr=HL)
        flags.Z = 1 if data[0] == 0 else 0
        flags.N = 0
        flags.H = 1
```

Detailed timing and pseudocode



```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3
if cb_mode and IR == 0x46: # example: BIT 0, (HL)
    Z = read_memory(addr=HL)
# M4/M1
flags.Z = 1 if Z[0] == 0 else 0
flags.N = 0
flags.H = 1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RES b, r: Reset bit (register)

Resets the bit b of the 8-bit register r to 0.

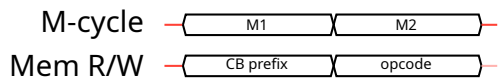
Opcode 0b10xxxxxx/various

Duration 2 machine cycles

Length 2 bytes: CB prefix + opcode

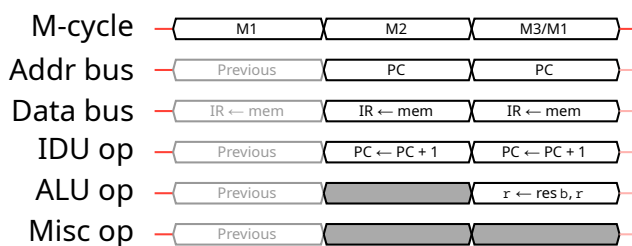
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
    if opcode == 0x80: # example: RES 0, B
        B = from_bits(7..1=B[7..1], 0=0)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0x80: # example: RES 0, B
    B = from_bits(7..1=B[7..1], 0=0)
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RES b, (HL): Reset bit (indirect HL)

Resets the bit b of the 8-bit data at the absolute address specified by the 16-bit register HL, to 0.

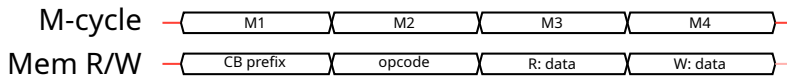
Opcode 0b10xxx110/variou

Duration 4 machine cycles

Length 2 bytes: CB prefix + opcode

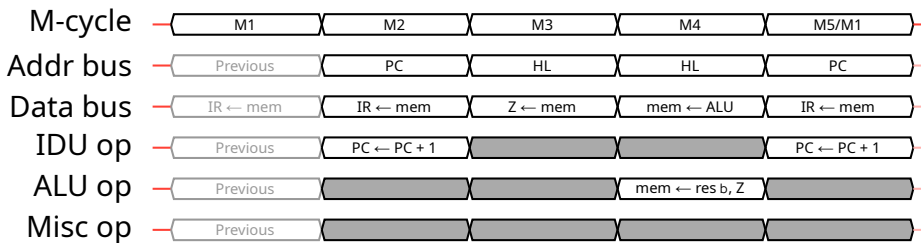
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
    if opcode == 0x86: # example: RES 0, (HL)
        data = read_memory(addr=HL)
        result = from_bits(7..1=data[7..1], 0=0)
        write_memory(addr=HL, data=result)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3
if cb_mode and IR == 0x86: # example: RES 0, (HL)
    Z = read_memory(addr=HL)
# M4
result = from_bits(7..1=Z[7..1], 0=0)
write_memory(addr=HL, data=result)
# M5/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SET b, r: Set bit (register)

Sets the bit b of the 8-bit register r to 1.

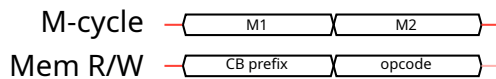
Opcode 0b11xxxxxx/ various

Duration 2 machine cycles

Length 2 bytes: CB prefix + opcode

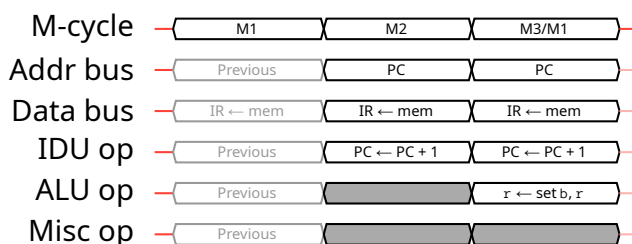
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
    if opcode == 0xC0: # example: SET 0, B
        B = from_bits(7..1=B[7..1], 0=1)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3/M1
if cb_mode and IR == 0xC0: # example: SET 0, B
    B = from_bits(7..1=B[7..1], 0=1)
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

SET b, (HL): Set bit (indirect HL)

Sets the bit *b* of the 8-bit data at the absolute address specified by the 16-bit register HL, to 1.

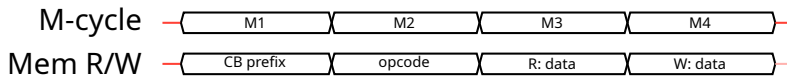
Opcode 0b11xxx110/variou

Duration 4 machine cycles

Length 2 bytes: CB prefix + opcode

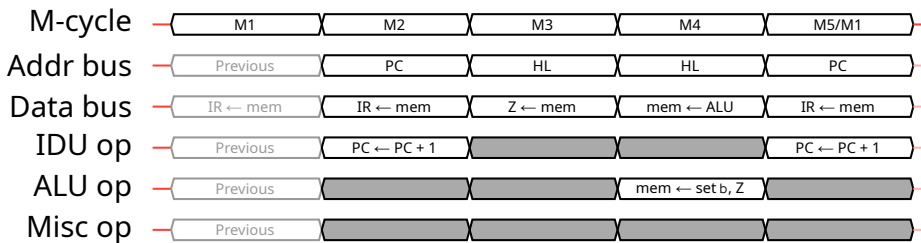
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xCB: # CB prefix
    opcode = read_memory(addr=PC); PC = PC + 1
    if opcode == 0xC6: # example: SET 0, (HL)
        data = read_memory(addr=HL)
        result = from_bits(7..1=data[7..1], 0=1)
        write_memory(addr=HL, data=result)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xCB:
    cb_mode = 1
    IR = fetch_cycle(addr=PC); PC = PC + 1
# M3
if cb_mode and IR == 0xC6: # example: SET 0, (HL)
    Z = read_memory(addr=HL)
# M4
result = from_bits(7..1=Z[7..1], 0=1)
write_memory(addr=HL, data=result)
# M5/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


6.7 Control flow instructions

JP nn: Jump

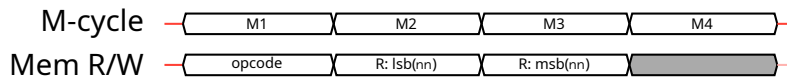
Unconditional jump to the absolute address specified by the 16-bit immediate operand nn.

Opcode 0b11000011/0xC3

Duration 4 machine cycles

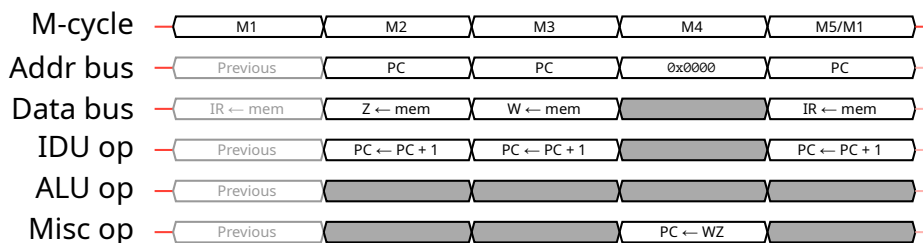
Length 3 bytes: opcode + LSB(nn) + MSB(nn) **Flags** -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xC3:
    nn_lsb = read_memory(addr=PC); PC = PC + 1
    nn_msb = read_memory(addr=PC); PC = PC + 1
    nn = unsigned_16(lsb=nn_lsb, msb=nn_msb)
    PC = nn
```

Detailed timing and pseudocode



```
# M2
if IR == 0xC3:
    Z = read_memory(addr=PC); PC = PC + 1
# M3
W = read_memory(addr=PC); PC = PC + 1
# M4
PC = WZ
# M5/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

JP HL: Jump to HL

Unconditional jump to the absolute address specified by the 16-bit register HL.

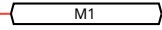
Opcode 0b11101001/0xE9

Duration 1 machine cycle

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

M-cycle 

Mem R/W 

```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0xE9:
```

```
    PC = HL
```

Detailed timing and pseudocode

M-cycle 

Addr bus 

Data bus 

IDU op 

ALU op 

Misc op 

```
# M2/M1
```

```
if IR == 0xE9:
```

```
    IR, intr = fetch_cycle(addr=HL); PC = HL + 1
```


Warning

In some documentation this instruction is written as JP [HL]. This is very misleading, since brackets are usually used to indicate a memory read, and this instruction simply copies the value of HL to PC.

JP cc, nn: Jump (conditional)

Conditional jump to the absolute address specified by the 16-bit operand nn, depending on the condition cc.

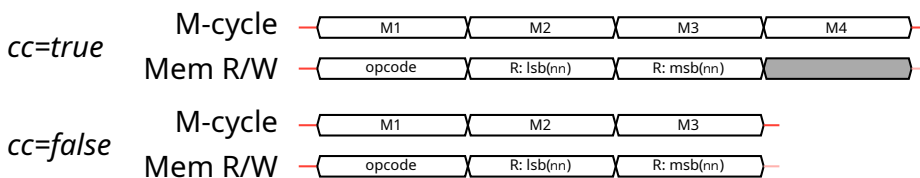
Note that the operand (absolute address) is read even when the condition is false!

Opcode 0b110xx010/variou

Duration 4 machine cycles (cc=true)
3 machine cycles (cc=false)

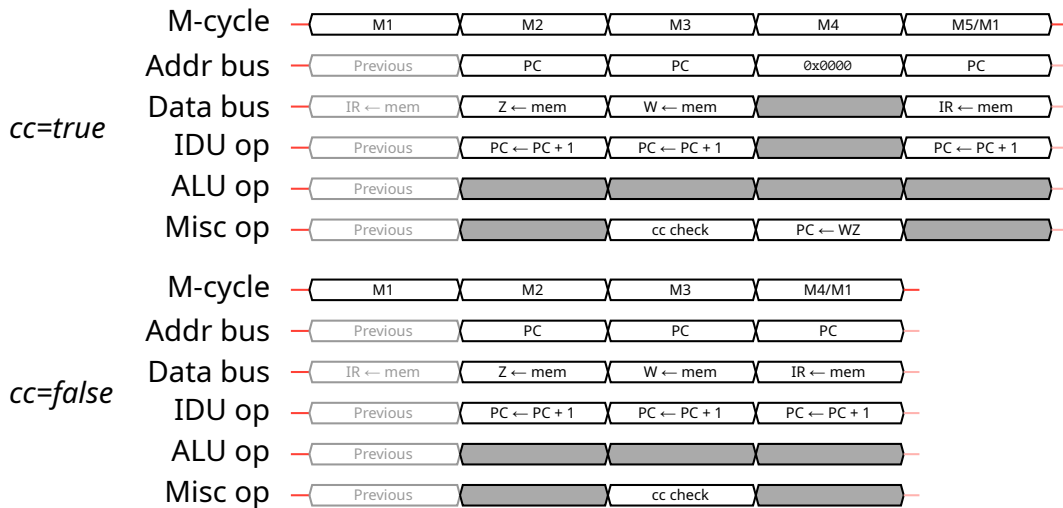
Length 3 bytes: opcode + LSB(nn) + MSB(nn) **Flags** -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xC2: # example: JP NZ, nn
    nn_lsb = read_memory(addr=PC); PC = PC + 1
    nn_msb = read_memory(addr=PC); PC = PC + 1
    nn = unsigned_16(lsb=nn_lsb, msb=nn_msb)
    if !flags.Z: # cc=true
        PC = nn
```

Detailed timing and pseudocode



```
# M2
if IR == 0xC2: # example: JP NZ, nn
    Z = read_memory(addr=PC); PC = PC + 1
# M3
W = read_memory(addr=PC); PC = PC + 1
if !flags.Z: # cc=true
    # M4
    PC = WZ
    # M5/M1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
else: # cc=false
    # M4/M1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

JR e: Relative jump

Unconditional jump to the relative address specified by the signed 8-bit operand e.

Opcode 0b00011000/0x18

Duration 3 machine cycles

Length 2 bytes: opcode + e

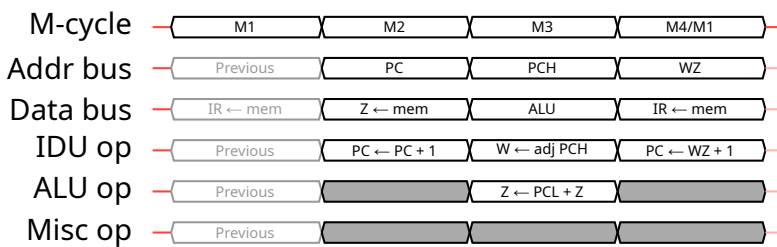
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0x18:
    e = signed_8(read_memory(addr=PC)); PC = PC + 1
    PC = PC + e
```

Detailed timing and pseudocode



```
# M2
if IR == 0x18:
    Z = read_memory(addr=PC); PC = PC + 1
# M3
Z_sign = bit(7, Z)
result, carry_per_bit = Z + lsb(PC)
Z = result
adj = 1 if carry_per_bit[7] and not Z_sign else
      -1 if not carry_per_bit[7] and Z_sign else
      0
W = msb(PC) + adj
# M4/M1
IR, intr = fetch_cycle(addr=WZ); PC = WZ + 1
```

JR cc, e: Relative jump (conditional)

Conditional jump to the relative address specified by the signed 8-bit operand *e*, depending on the condition *cc*.

Note that the operand (relative address offset) is read even when the condition is false!

Opcode 0b001xx000/variou

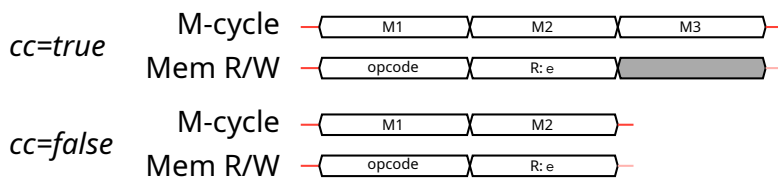
Duration 3 machine cycles (*cc*=true)

2 machine cycles (*cc*=false)

Length 2 bytes: opcode + *e*

Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

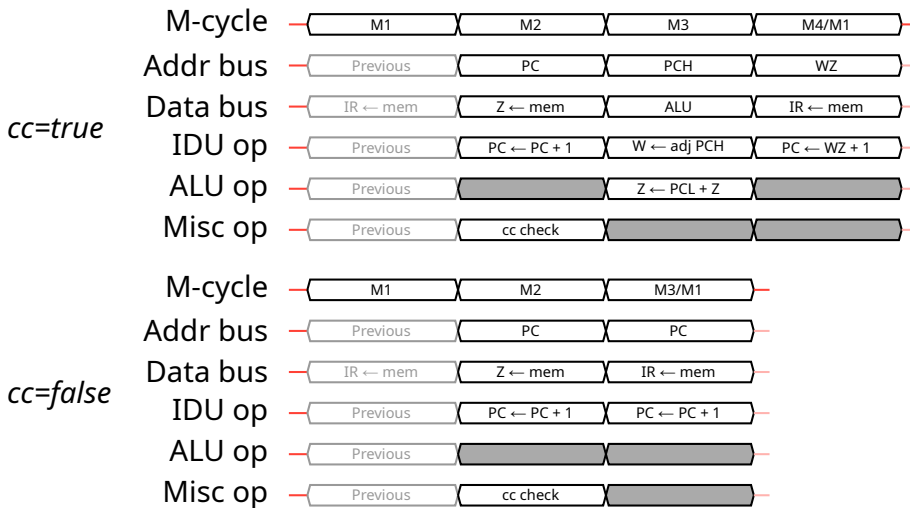
```
if opcode == 0x20:
```

```
    e = signed_8(read_memory(addr=PC)); PC = PC + 1
```

```
    if !flags.Z: # cc=true
```

```
        PC = PC + e
```

Detailed timing and pseudocode



```
# M2
if IR == 0x20:
    Z = read_memory(addr=PC); PC = PC + 1
    if !flags.Z: # cc=true
        # M3
        Z_sign = bit(7, Z)
        result, carry_per_bit = Z + lsb(PC)
        Z = result
        adj = 1 if carry_per_bit[7] and not Z_sign else
              -1 if not carry_per_bit[7] and Z_sign else
              0
        W = msb(PC) + adj
        # M4/M1
        IR, intr = fetch_cycle(addr=WZ); PC = WZ + 1
    else: # cc=false
        # M3/M1
        IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

CALL nn: Call function

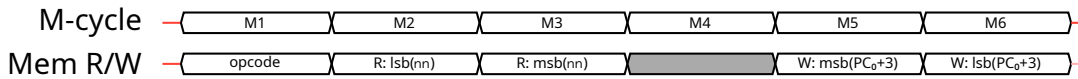
Unconditional function call to the absolute address specified by the 16-bit operand nn.

Opcode 0b11001101/0xCD

Duration 6 machine cycles

Length 3 bytes: opcode + LSB(nn) + MSB(nn) **Flags** -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0xCD:
```

```
    nn_lsb = read_memory(addr=PC); PC = PC + 1
```

```
    nn_msb = read_memory(addr=PC); PC = PC + 1
```

```
    nn = unsigned_16(lsb=nn_lsb, msb=nn_msb)
```

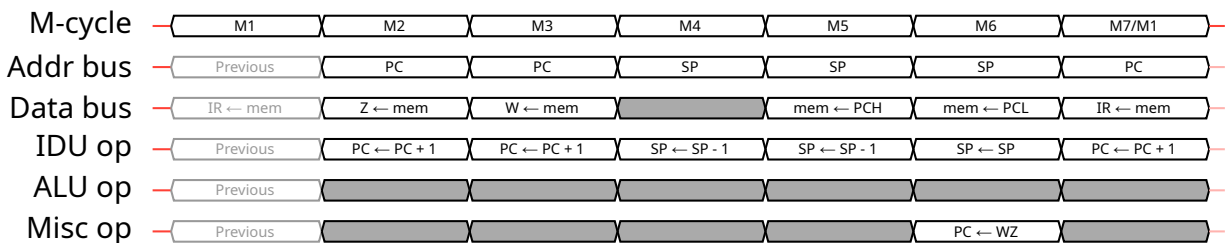
```
    SP = SP - 1
```

```
    write_memory(addr=SP, data=msb(PC)); SP = SP - 1
```

```
    write_memory(addr=SP, data=lsb(PC))
```

```
    PC = nn
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0xCD:
```

```
    Z = read_memory(addr=PC); PC = PC + 1
```

```
    # M3
```

```
    W = read_memory(addr=PC); PC = PC + 1
```

```
    # M4
```

```
    SP = SP - 1
```

```
    # M5
```

```
    write_memory(addr=SP, data=msb(PC)); SP = SP - 1
```

```
    # M6
```

```
    write_memory(addr=SP, data=lsb(PC)); PC = WZ
```

```
    # M7/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

CALL cc, nn: Call function (conditional)

Conditional function call to the absolute address specified by the 16-bit operand nn, depending on the condition cc.

Note that the operand (absolute address) is read even when the condition is false!

Opcode 0b110xx100/variou

Duration 6 machine cycles (cc=true)

3 machine cycles (cc=false)

Length 3 bytes: opcode + LSB(nn) + MSB(nn)

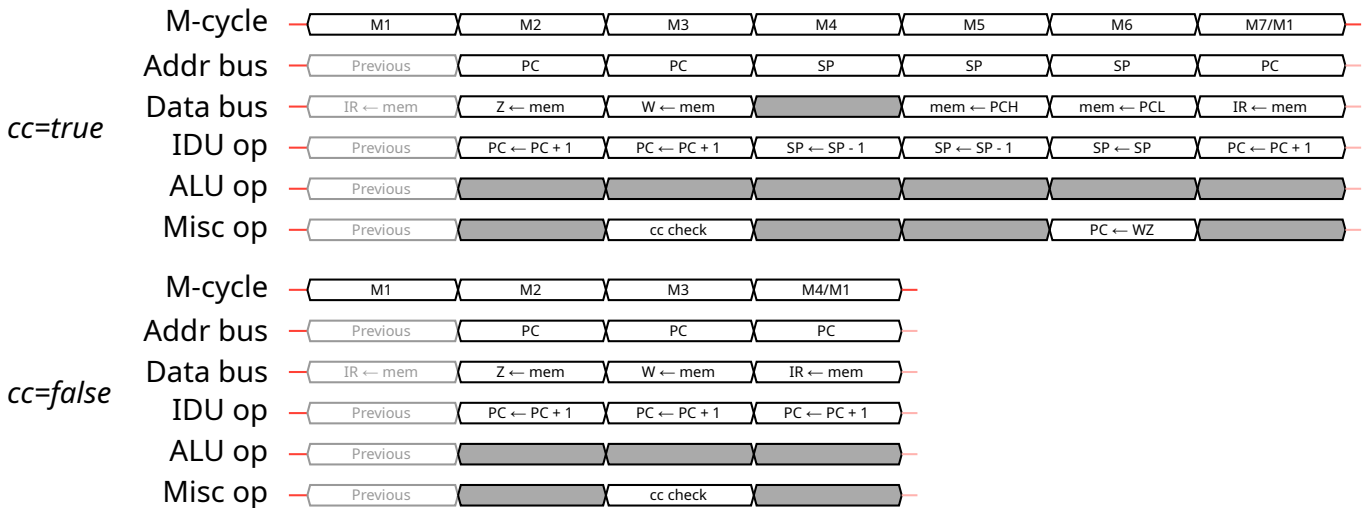
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xC4: # example: CALL NZ, nn
    nn_lsb = read_memory(addr=PC); PC = PC + 1
    nn_msb = read_memory(addr=PC); PC = PC + 1
    nn = unsigned_16(lsb=nn_lsb, msb=nn_msb)
    if !flags.Z: # cc=true
        SP = SP - 1
        write_memory(addr=SP, data=msb(PC)); SP = SP - 1
        write_memory(addr=SP, data=lsb(PC))
    PC = nn
```

Detailed timing and pseudocode



```
# M2
if IR == 0xC4: # example: CALL NZ, nn
    Z = read_memory(addr=PC); PC = PC + 1
# M3
W = read_memory(addr=PC); PC = PC + 1
if !flags.Z: # cc=true
    # M4
    SP = SP - 1
    # M5
    write_memory(addr=SP, data=msb(PC)); SP = SP - 1
    # M6
    write_memory(addr=SP, data=lsb(PC)); PC = WZ
    # M7/M1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
else: # cc=false
    # M4/M1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```


RET: Return from function

Unconditional return from a function.

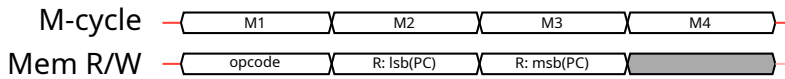
Opcode 0b11001001/0xC9

Duration 4 machine cycles

Length 1 byte: opcode

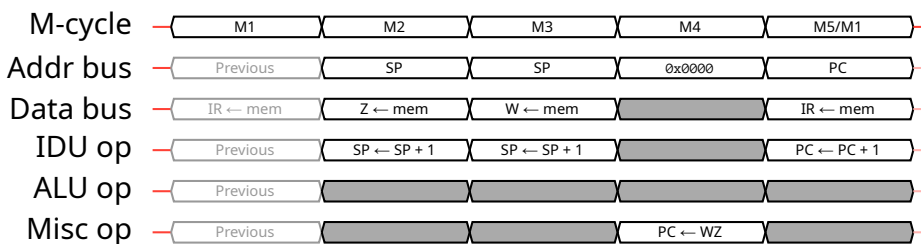
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xC9:
    lsb = read_memory(addr=SP); SP = SP + 1
    msb = read_memory(addr=SP); SP = SP + 1
    PC = unsigned_16(lsb=lsb, msb=msb)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xC9:
    Z = read_memory(addr=SP); SP = SP + 1
# M3
    W = read_memory(addr=SP); SP = SP + 1
# M4
    PC = WZ
# M5/M1
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RET cc: Return from function (conditional)

Conditional return from a function, depending on the condition cc.

Opcode 0b110xx000/variou

Duration 5 machine cycles (cc=true)
2 machine cycles (cc=false)

Length 1 byte: opcode

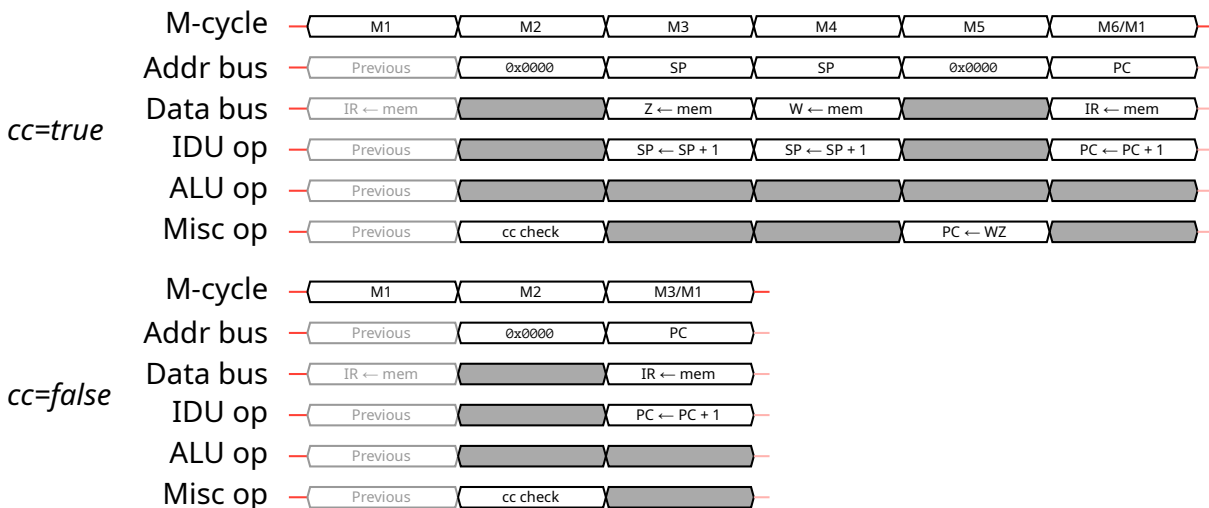
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xC0: # example: RET NZ
    if !flags.Z: # cc=true
        lsb = read_memory(addr=SP); SP = SP + 1
        msb = read_memory(addr=SP); SP = SP + 1
        PC = unsigned_16(lsb=lsb, msb=msb)
```

Detailed timing and pseudocode



```
# M2
if IR == 0xC0: # example: RET NZ
    if !flags.Z: # cc=true
        # M3
        Z = read_memory(addr=SP); SP = SP + 1
        # M4
        W = read_memory(addr=SP); SP = SP + 1
        # M5
        PC = WZ
        # M6/M1
        IR, intr = fetch_cycle(addr=PC); PC = PC + 1
    else: # cc=false
        # M3
        IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RETI: Return from interrupt handler

Unconditional return from a function. Also enables interrupts by setting IME=1.

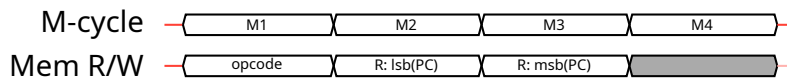
Opcode 0b11011001/0xD9

Duration 4 machine cycles

Length 1 byte: opcode

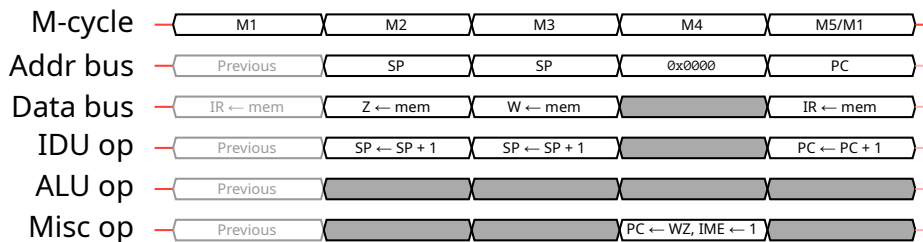
Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
if opcode == 0xD9:
    lsb = read_memory(addr=SP); SP = SP + 1
    msb = read_memory(addr=SP); SP = SP + 1
    PC = unsigned_16(lsb=lsb, msb=msb)
    IME = 1
```

Detailed timing and pseudocode



```
# M2
if IR == 0xD9:
    Z = read_memory(addr=SP); SP = SP + 1
# M3
W = read_memory(addr=SP); SP = SP + 1
# M4
PC = WZ; IME = 1
# M5/M1
IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

RST n: Restart / Call function (implied)

Unconditional function call to the absolute fixed address defined by the opcode.

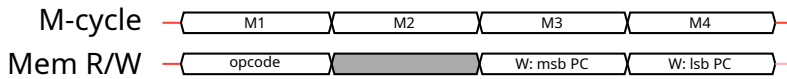
Opcode 0b11xxx111/ various

Duration 4 machine cycles

Length 1 byte: opcode

Flags -

Simple timing and pseudocode



```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0xDF: # example: RST 0x18
```

```
    n = 0x18
```

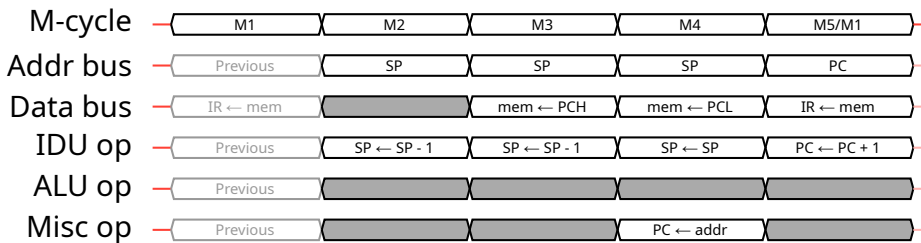
```
    SP = SP - 1
```

```
    write_memory(addr=SP, data=msb(PC)); SP = SP - 1
```

```
    write_memory(addr=SP, data=lsb(PC))
```

```
    PC = unsigned_16(lsb=n, msb=0x00)
```

Detailed timing and pseudocode



```
# M2
```

```
if IR == 0xDF: # example: RST 0x18
```

```
    SP = SP - 1
```

```
    # M3
```

```
    write_memory(addr=SP, data=msb(PC)); SP = SP - 1
```

```
    # M4
```

```
    write_memory(addr=SP, data=lsb(PC)); PC = 0x0018
```

```
    # M5/M1
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```

6.8 Miscellaneous instructions

HALT: Halt system clock

TODO

STOP: Stop system and main clocks

TODO

DI: Disable interrupts

Disables interrupt handling by setting IME=0 and cancelling any scheduled effects of the EI instruction if any.

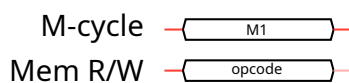
Opcode 0b11110011/0xF3

Duration 1 machine cycle

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

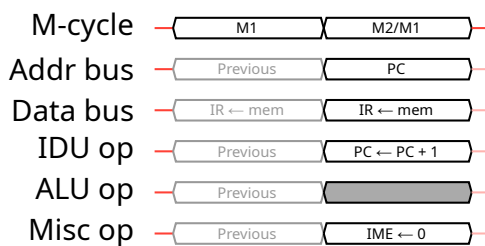


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0xF3:
```

```
    IME = 0
```

Detailed timing and pseudocode



```
# M2/M1
```

```
if IR == 0xF3:
```

```
    # interrupt checking is suppressed so fetch_cycle(..) is not used
```

```
    IR = read_memory(addr=PC); PC = PC + 1; IME = 0
```

EI: Enable interrupts

Schedules interrupt handling to be enabled after the next machine cycle.

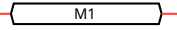
Opcode 0b11111011/0xFB

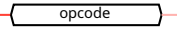
Duration 1 machine cycle

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

M-cycle 

Mem R/W 

```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0xFB:
```

```
    IME_next = 1
```

Detailed timing and pseudocode

M-cycle 

Addr bus 

Data bus 

IDU op 

ALU op 

Misc op 

```
# M2/M1
```

```
if IR == 0xFB:
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1; IME = 1
```

NOP: No operation

No operation. This instruction doesn't do anything, but can be used to add a delay of one machine cycle and increment PC by one.

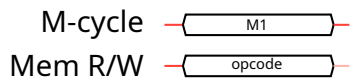
Opcode 0b00000000/0x00

Duration 1 machine cycle

Length 1 byte: opcode

Flags -

Simple timing and pseudocode

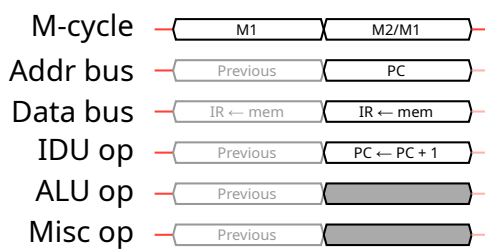


```
opcode = read_memory(addr=PC); PC = PC + 1
```

```
if opcode == 0x00:
```

```
    # nothing
```

Detailed timing and pseudocode



```
# M2/M1
```

```
if IR == 0x00:
```

```
    IR, intr = fetch_cycle(addr=PC); PC = PC + 1
```